The Ultimate DBMS & SQL Placement Preparation Guide

A Comprehensive 30+ Page Technical Interview Handbook

> **Domain Used Throughout:** A **Doctor Appointment Scheduling SaaS** (MediBook) with extensions into a **Live Video Interview Platform** (TalentCall) for variety. All SQL is PostgreSQL syntax.

Table of Contents

---

1. Core DBMS Architecture & Modeling

1.1 File Systems vs. DBMS

To understand why databases exist, you first have to feel the pain of managing data without one.

Imagine this scenario: You are building the MediBook appointment scheduling system in the early days. You decide to store all data in flat `.csv` files — one file per entity: `patients.csv`, `doctors.csv`, `appointments.csv`. At first it works. But here is what goes wrong almost immediately:

Data Redundancy & Inconsistency: Dr. Priya Sharma's phone number appears in `appointments.csv` (copied during booking), in `doctors.csv`, and in `referrals.csv`. When she changes her phone number, your developer updates only `doctors.csv` and

forgets the rest. Now the system has three different phone numbers for the same person. This is the **data inconsistency** problem.

Data Isolation: To find all appointments for patients older than 40 who have visited a cardiologist, you have to write custom code to open three files, parse them, join them manually, and filter them. Every new query requires new code. There is no query language.

Concurrent Access Problems: Two receptionists try to book the 10 AM slot with Dr. Kapoor simultaneously. Both read the file, both see it as free, both write "booked," and now the slot is double-booked. This is the classic **lost update** problem.

Security Issues: There is no fine-grained access control. A receptionist can see (and accidentally edit) doctors' salary information because everything is in the same files.

No Atomicity: The billing system crashes halfway through writing a transaction — the payment is deducted but the appointment is never confirmed. There is no rollback.

A **Database Management System (DBMS)** solves every one of these problems through:

Problem	DBMS Solution
Redundancy	Normalization + single source of truth
Inconsistency	Integrity constraints (FK, UNIQUE, CHECK)
Data Isolation	Declarative query language (SQL)
Concurrent access	Transaction management & locking
Security	Role-based access control (GRANT/REVOKE)
Atomicity	ACID transaction guarantees
No abstraction	Three-schema architecture

The key distinction is that a file system is a **dumb storage layer** — it only knows about bytes, files, and directories. A DBMS is an **intelligent data management layer** that understands the structure, relationships, meaning, and constraints on your data.

1.2 3-Schema Architecture & Data Independence

The ANSI/SPARC 3-schema architecture is the foundational blueprint of every modern DBMS. Its central insight is that the same data can and should be viewed differently by different stakeholders, and that these views should be insulated from each other.

Think of it like a multi-floor office building. The basement has the actual physical storage infrastructure (wiring, servers, disk arrays). The ground floor has a clean, standardized layout shared by the whole organization. And each office upstairs has its own customized view (the receptionist sees only scheduling, the billing team sees only payments).

The Three Schemas

1. External Schema (View Level) — "What users see"

This is the user-facing layer. Different user groups see different subsets or combinations of the data. In MediBook:

- A **patient** logs in and sees their own appointments, their prescriptions, and their doctor's contact info — nothing else.
- A **doctor** sees their schedule, their patients' medical histories, and lab results.
- A **hospital administrator** sees aggregate reports: total appointments per department, revenue per quarter, bed occupancy rates.

None of these users sees the raw underlying tables. They interact with **views** — virtual tables that present exactly the right slice of data. If the underlying table structure changes (say, the `patients` table is split into `patients` and `patient_demographics`), the external views can be updated to hide this change from the users.

2. Conceptual Schema (Logical Level) — "What the DBA sees"

This is the community view — a unified description of the entire database in terms of entities, attributes, relationships, and constraints. It describes **what** data is stored and how it is logically related, without caring about **how** it is physically stored.

In MediBook, the conceptual schema includes all tables: `patients`, `doctors`, `appointments`, `specializations`, `hospitals`, `prescriptions`, etc. — along with their relationships, primary keys, foreign keys, and data types. A DBA (Database Administrator) works at this level.

3. Internal Schema (Physical Level) — "What the storage engine sees"

This is the physical implementation. It describes how data is actually stored on disk: which file format, how records are laid out, which columns are indexed (and with what kind of index), how pages are organized, how data is compressed. A database storage engineer works here. Users and even most developers are completely unaware of this layer.

Data Independence — The Crown Jewel

The whole point of the 3-schema architecture is to achieve **data independence**, which means you can change one layer without forcing changes on the layers above it.

Physical Data Independence: You can change the internal schema (e.g., switch from a heap-organized table to a clustered B+ tree, or add a new index on `appointment_date`) without changing the conceptual schema or any application code. Applications continue to work exactly as before, just faster (or differently).

Logical Data Independence: You can change the conceptual schema (e.g., split the `patients` table into two tables) without changing the external schemas (views) or forcing application rewrites. This is harder to achieve in practice because it requires the views to be updated to compensate, but the architecture supports it.

This separation is what makes large enterprise databases maintainable. You can optimize storage without breaking applications. You can restructure the logical model without retraining users.

1.3 ER & EER Modeling

Entity-Relationship (ER) modeling is the blueprint stage of database design. Before writing a single line of SQL, you should have a crystal-clear ER diagram that maps out every entity, attribute, and relationship in your system. Think of it as the architect's floor plan before construction begins.

Core Concepts

Entity: A real-world object or concept with independent existence. In MediBook, `Patient`, `Doctor`, `Appointment`, `Hospital`, `Specialization`, and `Prescription` are all entities.

Attribute: A property that describes an entity.

- **Simple attribute:** Cannot be divided further. `first_name`, `date_of_birth`.
- **Composite attribute:** Made of sub-attributes. `full_address` = (`street`, `city`, `state`, `pincode`).
- **Derived attribute:** Computed from other attributes. `age` is derived from `date_of_birth`.
- **Multi-valued attribute:** Can hold multiple values. A doctor can have multiple `phone_numbers`.
- **Key attribute:** Uniquely identifies an entity. `patient_id` for Patient.

Relationship: An association between two or more entities. `Schedules` is a relationship between `Patient` and `Doctor` (via Appointment).

Cardinality (Participation Constraints):

- **One-to-One (1:1):** One doctor has one license. One license belongs to one doctor.

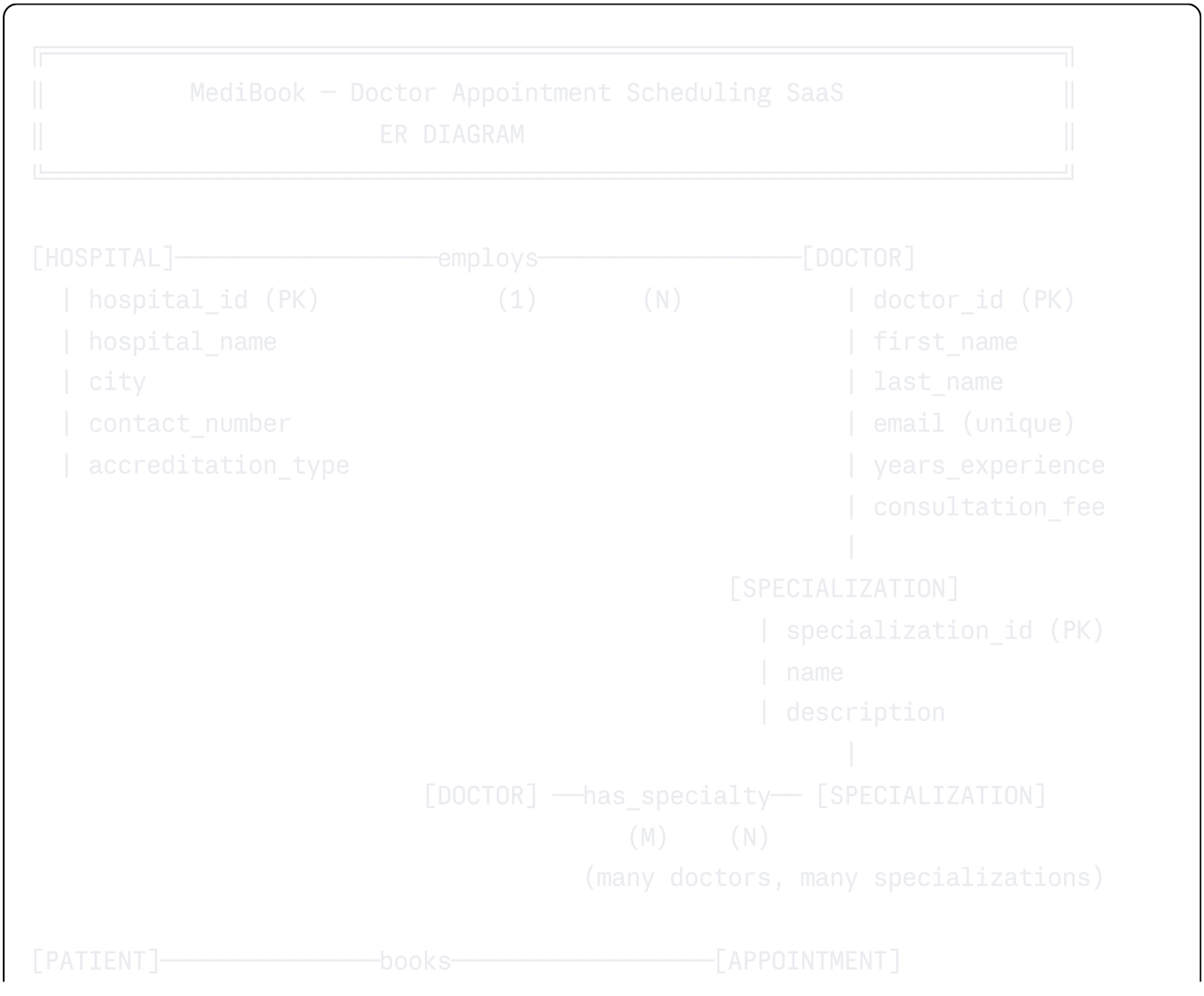
- **One-to-Many (1:N):** One doctor can have many appointments. Each appointment has exactly one doctor.
- **Many-to-Many (M:N):** Many patients can see many doctors (across multiple appointments). This always resolves to an intermediate table in the relational model.

Total vs. Partial Participation:

- **Total (double line):** Every entity must participate. Every `Appointment` MUST have a `Doctor`. An appointment cannot exist without a doctor.
- **Partial (single line):** Participation is optional. A `Doctor` may or may not have any appointments (a new doctor just joined).

Weak Entity: An entity that cannot be uniquely identified by its own attributes alone — it depends on a stronger (owner) entity. In MediBook, `Prescription` could be a weak entity: a prescription `(prescription_id = 1)` from appointment `A1` is different from `(prescription_id = 1)` from appointment `A2`. The prescription only makes sense in the context of its parent appointment. Its primary key is a **composite key** (`appointment_id` + `prescription_id`), where `appointment_id` is the **discriminator**.

MediBook ER Diagram (Text Representation)



| patient_id (PK)
| first_name
| last_name
| date_of_birth
| gender
| phone
| email
| address (composite)
├── street
├── city
├── state
└── pincode
(1)
(N)
| appointment_id (PK)
| appointment_date
| appointment_time
| status
(scheduled/completed/cancelled)
| reason_for_visit
| consultation_notes
| created_at
[APPOINTMENT] ──assigned_to── [DOCTOR]
(N)
(1)
[APPOINTMENT] ══generates══ [PRESCRIPTION] ← WEAK ENTITY (double rectangle)
(1)
(N)
| prescription_id (partial key)
| medication_name
| dosage
| duration_days
| refill_allowed
(PK = appointment_id + prescription_id)
[PATIENT] ──────── GENERALIZATION ────────
[INPATIENT]
| bed_number
| ward_name
| admission_date
[OUTPATIENT]
| preferred_time_slot
| telemedicine_eligible
[APPOINTMENT] ─── involves ─── [LAB_TEST] ← AGGREGATION
(M)
(N)
| test_id (PK)
| test_name
| result
| tested_on
LEGEND:
[ ] = Entity (rectangle)
[[ ]] = Weak Entity (double rectangle)
--- = Relationship
(1), (N), (M) = Cardinality

```
== = Total Participation
-- = Partial Participation
```

EER (Enhanced ER) Concepts

The Enhanced ER model extends basic ER with object-oriented concepts:

Generalization (Bottom-Up): You start with specific entities (`Inpatient`, `Outpatient`) and recognize they share common attributes, so you abstract them into a supertype (`Patient`). This is like saying "both of these are types of patients."

Specialization (Top-Down): You start with `Patient` and realize some patients have unique properties, so you create subtypes. A `Patient` who is admitted to a hospital bed becomes an `Inpatient` and gets `bed_number` and `ward_name`. A `Patient` seen in a clinic remains an `Outpatient` with `telemedicine_eligible`.

Constraint types in specialization:

- **Disjoint (d):** A patient can be EITHER an inpatient OR an outpatient, not both simultaneously.
- **Overlapping (o):** An entity can belong to multiple subclasses simultaneously (a doctor can be both a surgeon and a researcher).
- **Total:** Every patient must be in at least one subclass.
- **Partial:** A patient can exist without being in any subclass.

Aggregation: This handles the situation where a relationship itself participates in another relationship. In MediBook, an `Appointment` involving a `Patient` and `Doctor` may optionally involve `Lab_Tests`. The entire (Appointment–Patient–Doctor) relationship is treated as a higher-level entity that then connects to `Lab_Tests`. This avoids the awkwardness of connecting a relationship directly to an entity.

1.4 Conversion of ER to Relational Model

Converting an ER diagram to actual relational tables follows a systematic set of rules. Let's apply each rule to the MediBook schema.

Rule 1: Strong Entities → Tables

Each strong entity becomes a table. Simple attributes become columns. The key attribute becomes the primary key.

sql

```sql
CREATE TABLE hospitals (
    hospital_id   SERIAL PRIMARY KEY,
    hospital_name VARCHAR(200) NOT NULL,
    city          VARCHAR(100),
    contact_number VARCHAR(15),
    accreditation_type VARCHAR(50)
);

CREATE TABLE patients (
    patient_id     SERIAL PRIMARY KEY,
    first_name     VARCHAR(100) NOT NULL,
    last_name      VARCHAR(100) NOT NULL,
    date_of_birth  DATE NOT NULL,
    gender         CHAR(1) CHECK (gender IN ('M','F','O')),
    phone          VARCHAR(15),
    email          VARCHAR(150) UNIQUE,
    -- Composite attribute "address" decomposed into columns:
    street         VARCHAR(200),
    city           VARCHAR(100),
    state          VARCHAR(50),
    pincode        VARCHAR(10)
);

CREATE TABLE doctors (
    doctor_id         SERIAL PRIMARY KEY,
    first_name        VARCHAR(100) NOT NULL,
    last_name         VARCHAR(100) NOT NULL,
    email             VARCHAR(150) UNIQUE NOT NULL,
    years_experience  INT CHECK (years_experience >= 0),
    consultation_fee  NUMERIC(10,2),
    hospital_id       INT REFERENCES hospitals(hospital_id)  -- from "employs" 1
);
```

Rule 2: Composite Attributes → Flatten into Columns

As shown above, `address` (composite) is broken into `street`, `city`, `state`, `pincode` — individual atomic columns.

Rule 3: Multi-Valued Attributes → Separate Table

A doctor can have multiple phone numbers. This cannot be stored in a single column (that would violate 1NF). It becomes a separate table:

sql

```sql
CREATE TABLE doctor_phones (
    doctor_id    INT REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    phone_number VARCHAR(15) NOT NULL,
    phone_type   VARCHAR(20) DEFAULT 'mobile', -- mobile, clinic, home
    PRIMARY KEY (doctor_id, phone_number)
);
```

Rule 4: Weak Entities → Include Owner's PK as FK + Partial Key

sql

```sql
CREATE TABLE prescriptions (
    appointment_id    INT NOT NULL REFERENCES appointments(appointment_id) ON DEL
    prescription_id   INT NOT NULL,  -- partial key (relative to appointment)
    medication_name   VARCHAR(200) NOT NULL,
    dosage            VARCHAR(100),
    duration_days     INT,
    refill_allowed    BOOLEAN DEFAULT FALSE,
    PRIMARY KEY (appointment_id, prescription_id)
);
```

Rule 5: 1:N Relationships → FK on the "Many" Side

The `employs` relationship (Hospital 1 : N Doctor) → `hospital_id` FK goes in the `doctors` table (already done above). The `books` relationship (Patient 1 : N Appointment) → `patient_id` FK goes in `appointments`.

Rule 6: M:N Relationships → Junction Table

sql

```sql
-- Doctor ⇔ Specialization is M:N
CREATE TABLE specializations (
    specialization_id SERIAL PRIMARY KEY,
    name              VARCHAR(100) UNIQUE NOT NULL,
    description       TEXT
);

CREATE TABLE doctor_specializations (
    doctor_id            INT REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    specialization_id    INT REFERENCES specializations(specialization_id),
    certified_since      DATE,
    PRIMARY KEY (doctor_id, specialization_id)
);
```

Rule 7: Appointments Table (Central Junction)

```sql
CREATE TABLE appointments (
    appointment_id   SERIAL PRIMARY KEY,
    patient_id       INT NOT NULL REFERENCES patients(patient_id),
    doctor_id        INT NOT NULL REFERENCES doctors(doctor_id),
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status           VARCHAR(20) DEFAULT 'scheduled'
                       CHECK (status IN ('scheduled','completed','cancelled','no_
    reason_for_visit TEXT,
    consultation_notes TEXT,
    created_at       TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE (doctor_id, appointment_date, appointment_time) -- no double-booking
);
```

Rule 8: Generalization/Specialization → Choose a Strategy

There are three implementation strategies:

Strategy A — Single Table (Table per Hierarchy):

```sql
CREATE TABLE patients (
    patient_id          SERIAL PRIMARY KEY,
    -- ... shared columns ...
    patient_type        VARCHAR(20) CHECK (patient_type IN ('inpatient','outpatie
    -- Inpatient columns (NULL for outpatients):
    bed_number          VARCHAR(10),
    ward_name           VARCHAR(50),
    admission_date      DATE,
    -- Outpatient columns (NULL for inpatients):
    preferred_time_slot VARCHAR(20),
    telemedicine_eligible BOOLEAN
);
```

Pros: Simple, fast queries. *Cons:* Many NULLs, less clear schema.

Strategy B — Table per Subtype (Recommended for BCNF):

```sql
```

```
CREATE TABLE patients ( patient_id SERIAL PRIMARY KEY, first_name VARCHAR(100),
CREATE TABLE inpatients  ( patient_id INT PRIMARY KEY REFERENCES patients(patien
CREATE TABLE outpatients ( patient_id INT PRIMARY KEY REFERENCES patients(patien
```

Strategy C — Table per Concrete Type: No `patients` supertype table; inpatients and outpatients have completely separate tables with all shared columns duplicated. Violates DRY and causes update anomalies.

---

2. Relational Algebra & Calculus

2.1 Core Operators

Relational Algebra is a **procedural** query language — it tells the database engine exactly *how* to retrieve data, step by step. SQL, by contrast, is declarative (you say *what* you want). Under the hood, every SQL query the query optimizer processes is converted into a tree of relational algebra operations. Understanding RA makes you a much better SQL writer because you understand what the engine is actually doing.

Think of relational algebra like a factory assembly line. Data enters as raw tables (relations), passes through various operations (machines), and comes out as the result you want.

SELECT (σ) — The Row Filter

Syntax: **σ_condition(Relation)**

Selects rows that satisfy a condition. It does NOT eliminate duplicate columns. It works purely on the horizontal axis.

```
σ_status='scheduled'(appointments)
→ Returns only rows from appointments where status = 'scheduled'
```

SQL equivalent: `SELECT * FROM appointments WHERE status = 'scheduled';`

PROJECT (π) — The Column Filter

Syntax: **π_attr1, attr2, ...(Relation)**

Selects only specific columns and eliminates duplicates in the result (it is set-based). Works on the vertical axis.

```
π_first_name, last_name, email(patients)
→ Returns only the three named columns from patients
```

SQL equivalent: `SELECT DISTINCT first_name, last_name, email FROM patients;`

UNION (∪)

Combines rows from two relations. Both relations must be **union-compatible** (same number of attributes, same domains in corresponding positions). Eliminates duplicates.

```
π_email(patients) ∪ π_email(doctors)
→ All unique emails across both patients and doctors tables
```

SQL equivalent: `SELECT email FROM patients UNION SELECT email FROM doctors;`

SET DIFFERENCE (−)

Returns rows in the first relation that do NOT appear in the second. Relations must be union-compatible.

```
π_patient_id(patients) − π_patient_id(σ_status='completed'(appointments))
→ patient_ids of patients who have NEVER had a completed appointment
```

CARTESIAN PRODUCT (×)

Combines every row of one relation with every row of another. If relation R has m rows and S has n rows, R × S has m×n rows. This is the raw material for joins (a join = cartesian product + selection).

```
patients × doctors
→ Every possible (patient, doctor) combination
```

RENAME (ρ)

Gives a temporary name to a relation or its attributes. Essential when doing self-joins.

```
ρ_D1(doctors) × ρ_D2(doctors)
→ Two copies of the doctors table with different names
```

2.2 Derived Operators & Joins

Natural Join (⋈)

Automatically joins two relations on all columns with the same name, eliminating duplicate columns. It is shorthand for: Cartesian Product + Select (matching columns

equal) + Project (remove duplicate join columns).

```
appointments ⋈ doctors
→ Joins on doctor_id (common column), returns all columns without duplicating
doctor_id
```

Danger: Natural join can silently give wrong results if two columns happen to share a name but shouldn't be joined on. Always prefer explicit joins in SQL.

Equi Join

A join where the condition uses only equality. Almost all real-world joins are equi joins.

```
appointments ⋈_(appointments.doctor_id = doctors.doctor_id) doctors
```

Theta Join (⋈_θ)

A join with any comparison operator (=, ≠, <, >, ⩽, ⩾). For example, find all (patient, doctor) pairs where the patient's city matches the doctor's hospital's city:

```
patients ⋈_(patients.city = hospitals.city) hospitals
```

Outer Joins (Left, Right, Full)

Outer joins extend the natural join to include rows that don't have matching partners by padding with NULLs.

- **Left Outer Join (⟕):** All rows from the left relation, with NULLs for unmatched right-side columns.
- **Right Outer Join (⟖):** All rows from the right relation.
- **Full Outer Join (⟗):** All rows from both relations, with NULLs where there is no match.

```
patients ⟕_(p.patient_id = a.patient_id) appointments
→ Every patient, including those who have never booked an appointment
(appointment columns = NULL)
```

INTERSECTION (∩)

Returns rows common to both relations. Derivable as: R ∩ S = R − (R − S).

```
π_city(patients) ∩ π_city(hospitals)
→ Cities that have both patients AND hospitals in the system
```

DIVISION (÷)

The most underused but important operator. R ÷ S returns all tuples in R that are "associated with" every tuple in S.

Classic use case: "Find patients who have visited ALL doctors." If `appointments(patient_id, doctor_id) ÷ π_doctor_id(doctors)` → patients who have appointments with every doctor in the system.

SQL equivalent (using NOT EXISTS / EXCEPT approach):

```sql
SELECT DISTINCT a.patient_id FROM appointments a
WHERE NOT EXISTS (
    SELECT d.doctor_id FROM doctors d
    EXCEPT
    SELECT a2.doctor_id FROM appointments a2 WHERE a2.patient_id = a.patient_id
);
```

2.3 Five Complex RA Queries with SQL Equivalents

Query 1: Find the names of all patients who have a scheduled appointment with a cardiologist.

RA Expression:

```
π_p.first_name, p.last_name (
    σ_s.name='Cardiology' (
        patients ⋈_(p.patient_id=a.patient_id)
        appointments ⋈_(a.doctor_id=d.doctor_id)
        doctors ⋈_(d.doctor_id=ds.doctor_id)
        doctor_specializations ⋈_(ds.specialization_id=s.specialization_id)
        specializations
    ) where a.status = 'scheduled'
)
```

PostgreSQL:

```sql
```

```sql
SELECT DISTINCT p.first_name, p.last_name
FROM patients p
JOIN appointments a ON a.patient_id = p.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
WHERE s.name = 'Cardiology'
  AND a.status = 'scheduled';
```

Query 2: Find doctors who have NO completed appointments (doctors who have never treated a patient to completion).

RA Expression:

```
π_doctor_id(doctors) − π_doctor_id(σ_status='completed'(appointments))
```

PostgreSQL:

```sql
SELECT d.doctor_id, d.first_name, d.last_name
FROM doctors d
WHERE d.doctor_id NOT IN (
    SELECT DISTINCT a.doctor_id
    FROM appointments a
    WHERE a.status = 'completed'
);
-- Safer version using NOT EXISTS:
SELECT d.doctor_id, d.first_name, d.last_name
FROM doctors d
WHERE NOT EXISTS (
    SELECT 1 FROM appointments a
    WHERE a.doctor_id = d.doctor_id AND a.status = 'completed'
);
```

Query 3: Find all patients who have consulted at least two DIFFERENT doctors.

RA Expression:

```
π_a1.patient_id (
    ρ_A1(appointments) ⋈_(A1.patient_id=A2.patient_id AND
A1.doctor_id≠A2.doctor_id) ρ_A2(appointments)
)
```

PostgreSQL:

```sql
SELECT DISTINCT a1.patient_id, p.first_name, p.last_name
FROM appointments a1
JOIN appointments a2 ON a1.patient_id = a2.patient_id
                    AND a1.doctor_id <> a2.doctor_id
JOIN patients p ON p.patient_id = a1.patient_id;
-- Cleaner alternative:
SELECT patient_id FROM appointments
GROUP BY patient_id
HAVING COUNT(DISTINCT doctor_id) >= 2;
```

Query 4: Find the hospital(s) that employs doctors of EVERY specialization (division).

RA Expression:

```
π_hospital_id, specialization_id(doctors ⋈ doctor_specializations) ÷
π_specialization_id(specializations)
```

PostgreSQL (using NOT EXISTS double-negation for division):

```sql
SELECT h.hospital_id, h.hospital_name
FROM hospitals h
WHERE NOT EXISTS (
    SELECT s.specialization_id FROM specializations s
    WHERE NOT EXISTS (
        SELECT 1 FROM doctors d
        JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
        WHERE d.hospital_id = h.hospital_id
          AND ds.specialization_id = s.specialization_id
    )
);
```

Query 5: Find pairs of patients who share the same doctor AND live in the same city.

RA Expression:

```
π_p1.patient_id, p2.patient_id, p1.city (
    ρ_P1(patients) ⋈_(P1.city=P2.city AND P1.patient_id<P2.patient_id)
    ρ_P2(patients) ⋈_(P1 or P2's patient_id = a1.patient_id)
    ... [appointments join to find common doctor]
)
```

PostgreSQL:

```sql
SELECT DISTINCT p1.patient_id AS patient1_id,
                p2.patient_id AS patient2_id,
                p1.city,
                d.doctor_id
FROM patients p1
JOIN patients p2 ON p1.patient_id < p2.patient_id
                 AND p1.city = p2.city
JOIN appointments a1 ON a1.patient_id = p1.patient_id
JOIN appointments a2 ON a2.patient_id = p2.patient_id
                    AND a2.doctor_id = a1.doctor_id
JOIN doctors d ON d.doctor_id = a1.doctor_id;
```

2.4 Tuple & Domain Relational Calculus

While relational algebra is procedural (how to get data), **relational calculus** is declarative (what data we want). It's the mathematical foundation that proves SQL is relationally complete.

Tuple Relational Calculus (TRC)

In TRC, variables range over entire **tuples** (rows). A query is expressed as: `{ t | P(t) }` — "the set of all tuples t such that predicate P(t) is true."

Example: Find all patients whose city is 'Mumbai':

```
{ p | p ∈ patients ∧ p.city = 'Mumbai' }
```

Example: Find names of patients who have at least one appointment:

The existential quantifier ($\exists$) means "there exists at least one appointment a such that...".

The universal quantifier ($\forall$) handles "for all" conditions: Find patients who visited all doctors would use $\forall$.

Domain Relational Calculus (DRC)

In DRC, variables range over individual **domain values** (column values), not entire tuples.

Example: Find patient names and cities where city is 'Delhi':

```
{ <fn, ln, c> | <pid, fn, ln, dob, g, ph, email, street, c, state, pin> ∈
patients ∧ c = 'Delhi' }
```

DRC is the theoretical basis for **QBE (Query By Example)**, the graphical query interface used in older Microsoft Access.

Key difference: TRC variables are rows; DRC variables are individual field values. Both are equivalent in expressive power to relational algebra (Codd's theorem).

For interviews, remember: **SQL is based on TRC** — the `FROM` clause introduces tuple variables, `WHERE` is the predicate, and `SELECT` is the projection.

---

3. Normalization & Schema Refinement

3.1 Functional Dependencies & Armstrong's Axioms

Normalization is the science of designing tables so that data is stored efficiently without redundancy and without the structural problems that redundancy causes. But before you can normalize, you need to understand **Functional Dependencies (FDs)** — the formal language of data constraints.

What is a Functional Dependency?

A functional dependency X → Y means: "if you know the value of X, you can determine exactly one value of Y." Read as "X functionally determines Y" or "Y is functionally dependent on X."

Real analogy: In the MediBook system, if you know a `patient_id`, you can determine exactly one `date_of_birth`. You cannot have two patients with the same `patient_id` but different birth dates. So: `patient_id → date_of_birth`. This is a **valid FD**.

However, `city → doctor_id` is NOT a valid FD — many doctors can be in the same city. Knowing the city doesn't tell you which doctor.

Examples from MediBook:

```
patient_id → first_name, last_name, date_of_birth, phone, email, city
doctor_id  → first_name, last_name, email, consultation_fee, hospital_id
appointment_id → patient_id, doctor_id, appointment_date, appointment_time,
status
{doctor_id, appointment_date, appointment_time} → appointment_id  (composite
determinant)
```

Key Concepts

Superkey: A set of attributes that functionally determines ALL attributes of a relation.

Candidate Key: A minimal superkey — a superkey from which you cannot remove any attribute and still have it be a superkey. A relation may have multiple candidate keys.

Primary Key: One candidate key chosen as the main identifier (by convention/design).

Prime Attribute: Any attribute that is part of ANY candidate key.

Non-Prime Attribute: An attribute that is not part of any candidate key.

Closure of an Attribute Set: Given a set of FDs F, the closure of X (written X^+) is the set of all attributes that can be functionally determined starting from X.

Algorithm — Computing X^+:

```
Start: X⁺ = X
Repeat until no change:
    For each FD (Y → Z) in F:
        If Y ⊆ X⁺, then add Z to X⁺
```

Example: Given FDs: {A→BC, B→D, CD→E}, find $\{A\}^+$

```
Start: {A}⁺ = {A}
A→BC: A ⊆ {A}⁺, so add B,C → {A}⁺ = {A,B,C}
B→D: B ⊆ {A,B,C}, so add D → {A}⁺ = {A,B,C,D}
CD→E: CD ⊆ {A,B,C,D}, so add E → {A}⁺ = {A,B,C,D,E}
Result: {A}⁺ = {A,B,C,D,E} → A is a superkey (it determines all attributes)
```

Armstrong's Axioms (Inference Rules for FDs)

These are the foundational rules for deriving new FDs from a given set. They are **sound** (produce only valid FDs) and **complete** (can derive all valid FDs).

Axiom	Rule	Meaning
Reflexivity	If $Y \subseteq X$, then $X \to Y$	A set determines any of its subsets. `{patient_id, name} → patient_id`
Augmentation	If $X \to Y$, then $XZ \to YZ$	Adding the same attributes to both sides. If `doctor_id → name`, then `{doctor_id, city} → {name, city}`
Transitivity	If $X \to Y$ and $Y \to Z$, then $X \to Z$	Classic chain inference. `appointment_id → doctor_id` and `doctor_id → hospital_id` implies `appointment_id → hospital_id`

Derived Rules (provable from the three axioms above):

- **Union:** If X→Y and X→Z, then X→YZ
- **Decomposition:** If X→YZ, then X→Y and X→Z
- **Pseudotransitivity:** If X→Y and YW→Z, then XW→Z

Canonical Cover (Minimal Basis)

A canonical cover Fc of F is a minimal set of FDs that is equivalent to F (has the same closure). It has no redundant FDs and no redundant attributes on either side.

Algorithm to find canonical cover:

1. **Decompose RHS:** Replace any FD A→BC with two FDs A→B and A→C.
2. **Remove extraneous LHS attributes:** For each FD XA→Y, check if X→Y already derivable. If yes, remove A.
3. **Remove redundant FDs:** Check if each FD can be derived from the rest. If yes, remove it.

3.2 Anomalies

Anomalies are the bad things that happen when a poorly designed table tries to store too many concerns in one place. The classic example is a **single unnormalized table** that mixes entity data with relationship data.

The Problem Table

Suppose MediBook's early developer creates ONE table for everything:

appt_id	patient_name	patient_phone	doctor_id	doctor_name	hospital_name	speciali:
A1	Riya Menon	9876543210	D1	Dr. Sharma	Apollo	Cardiol
A2	Riya Menon	9876543210	D2	Dr. Patel	Fortis	Neurolo
A3	Amit Shah	9988776655	D1	Dr. Sharma	Apollo	Cardiol

Update Anomaly: Dr. Sharma changes her consultation fee from 800 to 1000. You must update EVERY row where she appears (A1, A3, and potentially hundreds more). If you miss even one row, the data becomes inconsistent. Different rows show different fees for the same doctor.

Insertion Anomaly: A new doctor, Dr. Mehta, joins Apollo hospital with Orthopedics specialization. But he hasn't had any appointments yet. Since `appt_id` is the primary key and is NOT NULL, you cannot insert Dr. Mehta's information — there's no appointment to hang it on. You either leave the doctor out or insert a fake appointment row with NULL appointment data (violating integrity).

Deletion Anomaly: If patient Amit Shah cancels appointment A3 and it gets deleted, and A3 was the ONLY appointment with Dr. Sharma at Apollo, you lose all information about Dr. Sharma and Apollo entirely — even though that information should exist independently of any appointment.

3.3 Normal Forms: 1NF through 5NF

Normalization is the process of decomposing tables to eliminate these anomalies. Each normal form addresses a specific type of problem. You move from lower to higher normal forms by decomposing tables into smaller, better-structured ones.

First Normal Form (1NF)

Rule: Every column must contain only **atomic (indivisible)** values. No repeating groups, no multi-valued columns, no arrays stored in a single column.

Violation example:

patient_id	patient_name	phone_numbers
P1	Riya Menon	9876543210, 9123456789

The `phone_numbers` column holds multiple values — a comma-separated list. This violates 1NF.

Fix: Either add a separate `patient_phones` table (preferred), or if the cardinality is small and truly fixed, use separate columns (`phone1`, `phone2`) — though the separate table approach is much cleaner.

Another violation: A column `appointment_times` that stores `"10:00, 11:00, 14:00"` in a single text field. Violates 1NF.

Fix for the problem table above (make it 1NF): Every cell must have one value. The table already has atomic values in each cell, so it IS in 1NF. The anomalies exist not because of 1NF violations but because of partial and transitive dependencies — which 2NF and 3NF address.

Second Normal Form (2NF)

Rule: The table must be in 1NF, AND every non-prime attribute must be **fully functionally dependent** on the ENTIRE primary key (not just part of it).

This rule only applies when the primary key is **composite** (made of multiple columns). If the primary key is a single column, 1NF automatically implies 2NF.

Violation example: Suppose we have a table tracking which doctors attended which appointments and which hospital they represented:

```
APPOINTMENT_DOCTOR_DETAIL(appt_id, doctor_id, doctor_name, hospital_name,
attendance_notes)
PK = {appt_id, doctor_id}
```

FDs:

- `{appt_id, doctor_id} → attendance_notes` ✅ Fully dependent on composite PK
- `doctor_id → doctor_name` ⚠️ **Partial dependency** — doctor_name depends only on part of the PK
- `doctor_id → hospital_name` ⚠️ **Partial dependency**

`doctor_name` and `hospital_name` depend on `doctor_id` alone, not on the full `{appt_id, doctor_id}` composite key. This is a 2NF violation.

Why is this bad? If Dr. Sharma appears in 100 appointment rows, her name is stored 100 times. Change her name → update 100 rows (update anomaly).

Fix (Decompose to 2NF):

```
APPOINTMENT_ATTENDANCE(appt_id, doctor_id, attendance_notes)    -- PK =
{appt_id, doctor_id}
DOCTORS(doctor_id, doctor_name, hospital_name)                  -- PK =
doctor_id
```

Third Normal Form (3NF)

Rule: The table must be in 2NF, AND no non-prime attribute must be **transitively dependent** on the primary key through another non-prime attribute.

In other words: no non-key column should determine another non-key column.

Violation example:

```
DOCTORS(doctor_id, doctor_name, hospital_id, hospital_name, hospital_city)
PK = doctor_id
```

FDs:

- `doctor_id → hospital_id` ✅ Direct dependency on PK
- `hospital_id → hospital_name, hospital_city` ⚠️ **Transitive dependency**: doctor_id → hospital_id → hospital_name

`hospital_name` is determined by `hospital_id`, not directly by `doctor_id`. Since `hospital_id` is a non-prime attribute, this is a transitive dependency and violates 3NF.

Why is this bad? If 50 doctors work at Apollo Hospital, "Apollo Hospital" is stored 50 times. Change the hospital name → 50 updates.

Fix (Decompose to 3NF):

```
DOCTORS(doctor_id, doctor_name, hospital_id)          -- PK = doctor_id
HOSPITALS(hospital_id, hospital_name, hospital_city)   -- PK = hospital_id
```

3NF Formal Definition: A relation R is in 3NF if for every non-trivial FD X→A in R, either:

1. X is a superkey (X determines the whole row), OR
2. A is a prime attribute (A is part of some candidate key)

Boyce-Codd Normal Form (BCNF)

Rule: The table must be in 3NF, AND for every non-trivial FD X→Y, **X must be a superkey**.

BCNF is stricter than 3NF. It removes the exception for prime attributes. The difference matters when there are **multiple overlapping candidate keys**.

Violation example (even though in 3NF): In MediBook, suppose a doctor can have multiple specializations, and for each specialization, exactly one senior doctor (coordinator) is assigned. Also, each doctor is coordinated by exactly one coordinator per specialization:

```
DOCTOR_SPECIALIZATION_COORDINATOR(doctor_id, specialization_id,
coordinator_id)
PK = {doctor_id, specialization_id}
Also a candidate key: {doctor_id, coordinator_id}  (each doctor has one
coordinator per spec)
```

FD: `coordinator_id → specialization_id` — a coordinator handles only one specialization.

Here, `coordinator_id` determines `specialization_id`, but `coordinator_id` is NOT a superkey (many doctors can share the same coordinator). This violates BCNF.

Fix: Decompose into:

```
DOCTOR_COORDINATOR(doctor_id, coordinator_id)          -- PK = {doctor_id,
coordinator_id}
COORDINATOR_SPECIALIZATION(coordinator_id, specialization_id)  -- PK =
coordinator_id
```

The 3NF vs. BCNF tradeoff: BCNF decomposition may not always preserve all FDs, while 3NF decomposition always preserves FDs. In practice, BCNF is preferred when possible, and 3NF is used as a fallback when BCNF would cause dependency loss.

Fourth Normal Form (4NF) — Multi-valued Dependencies

Rule: The table must be in BCNF, AND it should have no non-trivial **multi-valued dependencies** (MVDs) except those implied by a superkey.

A multi-valued dependency X →→ Y means "for each value of X, the set of Y values is independent of the other attributes." This happens when an entity has two independent one-to-many relationships.

Violation example:

```
DOCTOR_QUALIFICATIONS(doctor_id, degree, language_spoken)
```

Dr. Sharma has degrees {MBBS, MD} and speaks {English, Hindi, Marathi}. These are independent facts — which degrees she has is unrelated to which languages she speaks.

doctor_id	degree	language_spoken
D1	MBBS	English
D1	MBBS	Hindi
D1	MBBS	Marathi
D1	MD	English
D1	MD	Hindi
D1	MD	Marathi

You must store every combination (2 degrees × 3 languages = 6 rows for one doctor). This is redundant and causes update anomalies.

FD analysis: `doctor_id ↠ degree` (MVD) and `doctor_id ↠ language_spoken` (MVD) independently.

Fix (Decompose to 4NF):

```
DOCTOR_DEGREES(doctor_id, degree)
DOCTOR_LANGUAGES(doctor_id, language_spoken)
```

Now Dr. Sharma has 2 rows in `DOCTOR_DEGREES` and 3 rows in `DOCTOR_LANGUAGES` — total 5 rows instead of 6.

Fifth Normal Form (5NF) — Join Dependencies

Rule: The table must be in 4NF, AND it should not have any non-trivial **join dependencies** that are not implied by candidate keys.

5NF (also called Project-Join Normal Form) handles cases where decomposing into fewer than N tables would cause spurious tuples. It ensures that the original table can be reconstructed by joining the decomposed tables without generating incorrect extra rows.

5NF is extremely rare in practice and mostly a theoretical concern. In interviews, knowing it exists and understanding its purpose (avoiding spurious tuples from joins) is sufficient.

Normalization Summary Table

Normal Form	Eliminates	Key Rule
1NF	Repeating groups, non-atomic values	All attributes are atomic
2NF	Partial dependencies	Full dependency on entire PK
3NF	Transitive dependencies	No non-key → non-key
BCNF	All non-superkey determinants	Every determinant is a superkey
4NF	Multi-valued dependencies	No independent multi-valued facts
5NF	Join dependencies	Only trivial join dependencies

3.4 Lossless Join & Dependency-Preserving Decompositions

When you decompose a relation R into R1 and R2, you need to guarantee two properties:

Lossless Join Decomposition

A decomposition of R into {R1, R2} is **lossless** if joining R1 and R2 back together gives exactly R — no more (no spurious/extra rows) and no less.

Test (for two-way decomposition): The decomposition R→{R1, R2} is lossless if and only if:

- (R1 ∩ R2) → R1, OR

- (R1 ∩ R2) → R2

In plain English: the shared columns between R1 and R2 must be a superkey for at least one of R1 or R2.

Example: Decomposing `DOCTORS(doctor_id, doctor_name, hospital_id, hospital_name)` into:

- `R1 = DOCTORS(doctor_id, doctor_name, hospital_id)`
- `R2 = HOSPITALS(hospital_id, hospital_name)`
- R1 ∩ R2 = {hospital_id}
- hospital_id → hospital_name (hospital_id is PK of R2) ✅ Lossless!

Example of LOSSY decomposition:

- `R1 = (patient_id, doctor_id)`
- `R2 = (doctor_id, appointment_date)`
- R1 ∩ R2 = {doctor_id}
- `doctor_id` does NOT determine `patient_id` (many patients per doctor)
- `doctor_id` does NOT determine `appointment_date`
- Result: joining R1 and R2 produces spurious rows — every patient gets paired with every date for that doctor. LOSSY! ❌

Dependency-Preserving Decomposition

A decomposition is **dependency-preserving** if every FD in the original set F can be checked using only the individual decomposed tables, without needing to join them.

Why this matters: If a crucial FD can only be verified after joining tables, the database engine must perform an expensive join every time a row is inserted or updated to verify the constraint. This kills performance.

Example: Original FDs: `{A→B, B→C}`. Decompose into:

- R1(A, B) — preserves A→B ✅
- R2(B, C) — preserves B→C ✅ This decomposition is dependency-preserving ✅

Trade-off: BCNF always gives lossless decomposition but may NOT preserve all dependencies. 3NF always gives both lossless AND dependency-preserving decomposition. This is why 3NF is often preferred as the "safe" normalization target in production systems.

3.5 When to Denormalize

Normalization is the gold standard for data integrity, but sometimes performance requirements force you to consciously introduce controlled redundancy. This is called **denormalization**, and it's a deliberate, informed decision — not a design mistake.

When denormalization is justified:

Read-heavy analytical workloads: In MediBook's reporting dashboard, a query that asks "show the monthly revenue by hospital by specialization" requires joining `appointments`, `doctors`, `doctor_specializations`, `specializations`, and `hospitals` — five tables. If this query runs millions of times per day, pre-computing a denormalized `appointment_summary` table with `hospital_name` and `specialization_name` already embedded can reduce query time from seconds to milliseconds.

OLAP vs. OLTP: Online Transaction Processing (OLTP) systems (booking appointments, updating patient records) benefit from normalization. Online Analytical Processing (OLAP) systems (BI dashboards, reports) benefit from denormalization — specifically the **star schema** and **snowflake schema** used in data warehouses.

Common denormalization patterns:

- **Storing derived values:** Pre-computing `total_appointments` on the `patients` table to avoid a COUNT query every time.
- **Duplicating columns for joins:** Storing `doctor_name` directly in the `appointments` table even though it exists in `doctors`.
- **Materialized views:** PostgreSQL's `CREATE MATERIALIZED VIEW` lets you pre-compute expensive joins and refresh them on a schedule.

The rule: Always start fully normalized. Denormalize only after profiling shows a genuine performance problem, document the decision, and ensure the redundant data is kept consistent via triggers or application logic.

---

4. Storage & Indexing Engine

4.1 Primary, Secondary & Clustered Indexing

An index is exactly like the index at the back of a textbook. Without it, to find a topic, you'd read every single page from start to end (a full table scan). With an index, you go directly to the right page.

In database terms, an index is a separate data structure that maps column values to the physical location (block address / row ID) of the corresponding records, enabling fast lookup without scanning the entire table.

Dense vs. Sparse Indexes

Dense index: One index entry per record in the data file. Every key value in the table has a corresponding pointer in the index. Requires more space but can answer queries directly from the index.

Sparse index: One index entry per block (page) of the data file. Takes much less space but requires the data file to be sorted — you use the index to find the right block, then scan that block. Typically used with primary (clustered) indexes.

Primary Index (Clustering Index)

A **primary index** is built on the **ordering key field** of an ordered data file. The data file itself is physically sorted by this field. Because the data is already sorted, you can use a sparse index — one entry per disk block.

In PostgreSQL, a table with a `CLUSTER` command applied to it uses a clustered index. The table rows are physically reordered to match the index order.

```sql
-- Create a clustered index on appointments by appointment_date
CREATE INDEX idx_appts_date ON appointments(appointment_date);
CLUSTER appointments USING idx_appts_date;
-- Now appointment rows are physically ordered by date on disk
```

Key property: Because the data is sorted by the index key, range queries (`WHERE appointment_date BETWEEN '2024-01-01' AND '2024-12-31'`) are extremely efficient — you find the starting block and read forward sequentially.

Limitation: A table can have only ONE clustered index (the data can only be sorted one way at a time).

Secondary Index (Non-Clustering Index)

A **secondary index** is built on any field that is NOT the primary ordering field. The index entries are dense (one per record) and contain pointers to the actual physical locations of the records. The data file itself is NOT sorted by this field.

In PostgreSQL, every non-primary index you create is a secondary index.

```sql
```

```
-- Secondary index for finding appointments by status quickly
CREATE INDEX idx_appts_status ON appointments(status);

-- Secondary index for doctor-specific appointment lookups
CREATE INDEX idx_appts_doctor ON appointments(doctor_id);
```

Secondary index on a non-key field: Multiple records can have the same status ('scheduled', 'completed'). So the secondary index either:

1. Points to a bucket (list) of pointers for each status value, or
2. Creates one dense index entry per record (with duplicates in the index)

The double indirection cost: Secondary index → index entry → actual row on disk. If the data is not sorted, accessing multiple rows from a secondary index scan causes many random disk seeks (bad for HDDs, less bad for SSDs).

Composite Indexes

sql

```
-- Composite index for queries that filter by both doctor AND date
CREATE INDEX idx_appts_doctor_date ON appointments(doctor_id, appointment_date);
```

Prefix rule: A composite index on (A, B, C) can be used for queries filtering on A, or A+B, or A+B+C — but NOT for queries filtering only on B or only on C. The leftmost prefix must be used.

Covering Index

An index that contains ALL the columns a query needs — so the query can be answered entirely from the index without touching the main table at all. This is called an **index-only scan** in PostgreSQL.

sql

```
-- If this is a frequent query:
SELECT doctor_id, appointment_date, status FROM appointments WHERE doctor_id = 5;

-- A covering index includes all needed columns:
CREATE INDEX idx_covering ON appointments(doctor_id) INCLUDE (appointment_date, s
-- PostgreSQL now never has to touch the appointments table for this query
```

4.2 B-Trees vs. B+ Trees (Deep Dive)

This is one of the most important topics in storage engine design and appears in senior-level interviews frequently. Almost every production database (PostgreSQL, MySQL InnoDB, Oracle, SQL Server) uses **B+ Trees** as the primary index structure. Understanding *why* requires understanding both structures.

The Problem B-Trees Solve

Imagine your `appointments` table has 100 million rows. A binary search tree (BST) would have ~$\log_2(100{,}000{,}000) \approx 27$ levels. But here's the problem: **each level requires a disk I/O operation** (because tree nodes are spread across different disk blocks). That's 27 disk reads for one lookup.

B-Trees and B+ Trees are **balanced, multi-way search trees** designed to minimize the number of disk I/O operations by making each node very wide (holding many keys), which drastically reduces tree height. A B+ Tree node can hold hundreds of keys, reducing height to 3-4 levels even for billions of rows.

B-Tree Structure

In a B-Tree (Balanced Tree) of order m:

- Each internal node holds between ⌈m/2⌉ - 1 and m-1 **keys**
- Each internal node has between ⌈m/2⌉ and m **child pointers**
- **Each node (internal and leaf) stores both the key AND the actual data record pointer**
- All leaf nodes are at the same depth (balanced)

```
Example B-Tree (order 3, max 2 keys per node):

                    [30 | 70]
                   /    |    \
            [10|20]  [40|60]  [80|90]
            /  | \    /  | \    /  | \
          data data  data ...  data data
```

Critical property of B-Tree: Data pointers can appear at ANY level — in root, internal nodes, or leaf nodes.

B+ Tree Structure

In a B+ Tree:

- **Internal nodes store ONLY keys (no data pointers)** — they are purely navigation guides

- **ALL data pointers are in the leaf nodes ONLY**
- **Leaf nodes are linked together in a doubly-linked list** (this is the game-changing addition)
- Internal nodes can thus hold MORE keys (since they don't store data), making the tree even wider and shorter

```
Example B+ Tree (order 4):

INTERNAL NODES (navigation only — keys are routing guides):
                      [40 | 70]
                     /    |    \
            [20|30]    [50|60]    [80|90]

LEAF NODES (all data here, linked as a list):
[10|data] ↔ [20|data] ↔ [30|data] ↔ [40|data] ↔ [50|data] ↔ ...
 ↑ All leaves linked in sorted order via sibling pointers
```

Why B+ Trees Are Superior to B-Trees for Databases

Reason 1: Higher Fanout, Shorter Tree

Because internal nodes in B+ Trees store ONLY keys (not data records, which are much larger), each internal node can hold far more keys. More keys per node = more children per node = higher **fanout** = fewer levels in the tree = fewer disk I/O operations.

For a practical example: if a B-Tree internal node holds 100 (key, data_pointer) pairs, a B+ Tree internal node might hold 500 keys (because each entry is smaller — just a key, no data pointer). For 100 million records:

- B-Tree height $\approx \log_{100}(100M) \approx 4$ levels
- B+ Tree height $\approx \log_{500}(100M) \approx 3$ levels

That's 25% fewer disk seeks on every single key lookup, at any scale.

Reason 2: Sequential Range Scans Are Blazing Fast

This is the most critical advantage for databases. Consider the query:

```sql
SELECT * FROM appointments WHERE appointment_date BETWEEN '2024-06-01' AND '2024-
```

With a **B-Tree**, the data records are scattered throughout the tree at every level. To find all dates in the range, you must traverse the tree for EACH matching date — an in-order traversal that bounces up and down the tree.

With a **B+ Tree**, ALL leaf nodes contain the data pointers AND are linked in a doubly-linked list sorted by key value. The algorithm is:

1. Traverse the tree to find the leaf node for `2024-06-01` — O(log n)
2. **Walk the linked list forward** until you pass `2024-06-30` — pure sequential I/O!

Sequential I/O is 10-100x faster than random I/O on HDDs, and even on SSDs the sequential access pattern maximizes throughput. This is why B+ Trees dominate: databases are overwhelmingly used for range queries, not just single-key lookups.

Reason 3: Better Cache Utilization

Because internal nodes are dense (keys only), the entire upper levels of a B+ Tree often fit in the **buffer pool** (database cache) after the first few accesses. PostgreSQL's shared_buffers hold frequently accessed pages. In a well-tuned PostgreSQL instance, the root and second level of every major index are almost always in cache, meaning most queries need only 1-2 actual disk reads for the leaf level.

Reason 4: Consistent Performance

In a B-Tree, a lookup for a key stored in the root node takes O(1) disk reads, while a key at a leaf takes O(log n). In a B+ Tree, EVERY key lookup takes exactly the same path length (root → leaf), giving perfectly consistent, predictable performance. Databases need predictability.

Reason 5: Deletion Simplicity

Deleting from a B-Tree sometimes requires restructuring internal nodes (because data lives there). Deleting from a B+ Tree only ever modifies leaf nodes; the internal nodes remain as routing guides even if the actual data is gone. Rebalancing is still needed to maintain the minimum fill factor, but it's simpler and less frequent.

Summary Comparison

Aspect	B-Tree	B+ Tree
Data storage	Internal + leaf nodes	Leaf nodes ONLY
Range queries	Slow (tree traversal)	Fast (linked list scan)
Key lookup	Variable depth O(log n)	Consistent O(log n)
Fanout / tree height	Lower fanout, taller	Higher fanout, shorter
Internal node size	Larger (stores data)	Smaller (keys only)
Cache efficiency	Lower	Higher

Aspect	B-Tree	B+ Tree
Used in practice	Rarely for databases	PostgreSQL, MySQL InnoDB, Oracle

4.3 Hash Indexing & Bitmap Indexes

Hash Indexes

Hash indexes use a hash function to map key values to bucket addresses. They provide **O(1) average-case point lookups** — dramatically faster than B+ Tree's O(log n) for exact equality searches.

```sql
-- PostgreSQL supports hash indexes:
CREATE INDEX idx_patient_email_hash ON patients USING HASH (email);
-- Lightning fast for: SELECT * FROM patients WHERE email = 'riya@example.com';
```

Fatal weakness: Hash indexes are completely useless for range queries. There is no ordering in a hash table. `WHERE appointment_date > '2024-01-01'` cannot use a hash index at all — the database falls back to a full table scan.

Also weak at: LIKE queries, ORDER BY, inequality conditions. Use hash indexes ONLY when you know all queries will be exact equality lookups.

Bitmap Indexes

Bitmap indexes are specialized for **low-cardinality columns** — columns with few distinct values (gender, status, blood_type, boolean flags).

For a column `status` with values {scheduled, completed, cancelled, no_show}, a bitmap index creates one bit-vector per distinct value:

```
Appointment rows:    A1  A2  A3  A4  A5  A6  A7  A8
scheduled bitmap:     1   0   1   0   0   1   0   1
completed bitmap:     0   1   0   1   0   0   1   0
cancelled bitmap:     0   0   0   0   1   0   0   0
```

Querying `WHERE status = 'scheduled'` scans the 'scheduled' bitmap and retrieves rows where the bit is 1.

Key advantage: Combining multiple conditions (`WHERE status='scheduled' AND gender='F'`) is extremely fast — you just AND the two bitmaps together using CPU bit operations. This is massively faster than traditional index lookups.

PostgreSQL doesn't have a dedicated BITMAP INDEX type (unlike Oracle), but it performs **bitmap index scans** internally when combining multiple B-Tree indexes.

---

5. Transaction Management & Concurrency Control

5.1 ACID Properties

ACID is the cornerstone promise that a reliable DBMS makes about every transaction. Without ACID guarantees, a database storing medical records, financial data, or bookings would be completely untrustworthy.

A **transaction** is a unit of work — a sequence of one or more database operations (reads/writes) that must be executed as an indivisible whole. In MediBook, booking an appointment involves multiple steps: check doctor availability, reserve the slot, create the appointment record, deduct a consultation fee hold, send a confirmation. All of these must succeed together or fail together.

Atomicity — "All or Nothing"

Every operation in a transaction either ALL succeed or ALL are undone. There is no partial execution visible to the outside world.

Example: Riya Menon books an appointment. The system:

1. Inserts a row into `appointments` (payment_status = 'pending')
2. Deducts ₹800 from her wallet
3. Updates the doctor's availability slot to 'booked'

If step 3 crashes (power failure, network timeout), atomicity ensures steps 1 and 2 are automatically rolled back. The database returns to its pre-booking state. Riya is not charged, and the appointment doesn't exist in a half-created state.

How it's implemented: The DBMS uses an **undo log** — before modifying any data, the original value is written to a log. If the transaction fails, the undo log is used to restore the original values.

Consistency — "Validity Preserved"

A transaction moves the database from one **valid state** to another valid state, respecting all defined rules (constraints, triggers, cascades).

Example: The `appointments` table has a UNIQUE constraint on `(doctor_id, appointment_date, appointment_time)`. The consultation_fee cannot be negative. If any transaction would violate these rules, it is rejected entirely (rolled back), and the database stays in its previous valid state.

Isolation — "Transactions Don't See Each Other Mid-Execution"

Concurrent transactions execute as if they are the only transaction running. Intermediate states of one transaction are not visible to other transactions.

Example: Two patients simultaneously try to book Dr. Kapoor's 11 AM slot on Monday. Without isolation, both might read the slot as available, both create appointment rows, and now the doctor has two appointments at the same time. With proper isolation, one transaction completes first (slot marked as booked), and the second transaction then reads the slot as unavailable and is prevented from double-booking.

The spectrum: Full isolation is expensive (it requires locking or versioning everything). SQL defines four isolation levels that trade off isolation strength against performance — covered in detail in Section 5.5.

Durability — "Committed Changes Survive Crashes"

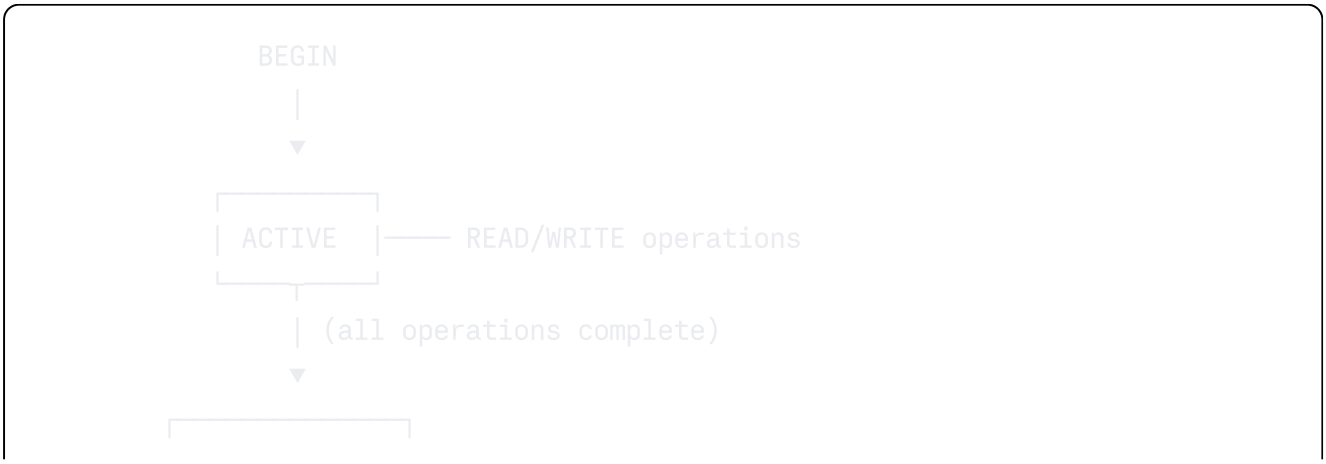
Once a transaction is committed, its changes are permanent and will survive even a complete system crash (power failure, OS crash, hardware failure).

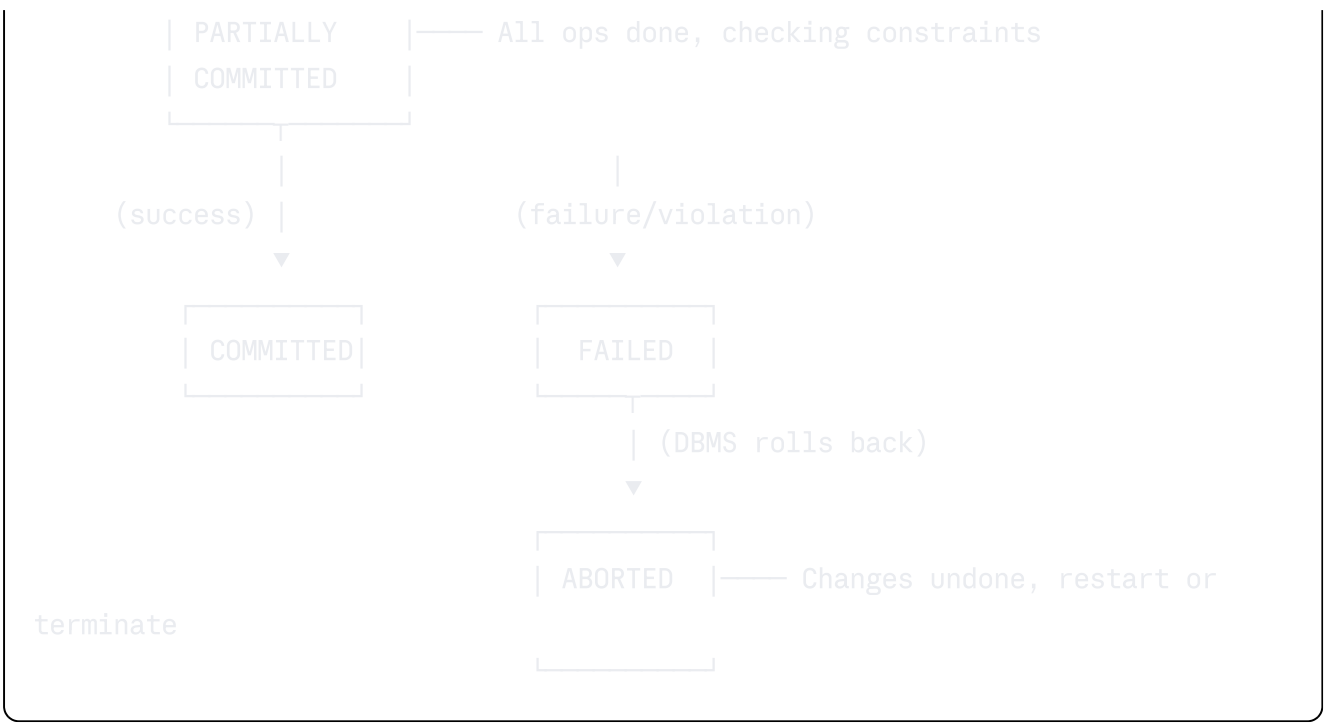
How it's implemented: The DBMS uses a **write-ahead log (WAL)**. Before any data is written to the actual data files, the change is first written to the WAL (a separate sequential log file). Even if the system crashes right after a commit, when the system restarts, it replays the WAL to bring the data files up to the committed state.

This is why `COMMIT` in PostgreSQL doesn't return until the WAL has been flushed to durable storage (disk sync).

5.2 Transaction States & Schedules

Transaction State Machine





A transaction that reaches **ABORTED** can either be restarted (if the failure was transient, e.g., a deadlock) or killed (if the failure was due to a logic error).

Schedules

When multiple transactions run concurrently, the DBMS interleaves their operations. The resulting interleaving is called a **schedule**.

Serial schedule: T1 runs completely, then T2 runs completely (or vice versa). No interleaving. Always correct but prevents concurrency — unacceptable for performance.

Concurrent schedule: Operations from multiple transactions are interleaved. The goal is to find concurrent schedules that are "equivalent" to some serial schedule — meaning they produce the same result.

5.3 Serializability

Conflict serializability is the most important concept in concurrency theory. A schedule is serializable if it is equivalent to some serial schedule.

Conflicting Operations

Two operations from different transactions **conflict** if they:

1. Access the same data item, AND
2. At least one of them is a WRITE

Operation pair	Conflict?
T1 Read(X), T2 Read(X)	No — two reads never conflict
T1 Read(X), T2 Write(X)	Yes — write changes what T1 reads
T1 Write(X), T2 Read(X)	Yes — same as above, different order
T1 Write(X), T2 Write(X)	Yes — overwrites are order-dependent

Conflict Serializability — Precedence Graph

Build a directed graph where:

- Nodes are transactions
- Draw edge T1 → T2 if an operation in T1 **conflicts with** and appears **before** a conflicting operation in T2

The schedule is **conflict-serializable** if and only if the precedence graph has **no cycles**.

Example:

```
T1: R(A) W(A)       R(B) W(B)
T2:         R(A) W(A)       R(B) W(B)

Conflicts:
T1 W(A) before T2 R(A) → edge T1→T2
T1 W(A) before T2 W(A) → edge T1→T2
T1 R(B) before T2 W(B) → edge T1→T2
T1 W(B) before T2 W(B) → edge T1→T2

Precedence graph: T1 → T2 (no cycle) → SERIALIZABLE ✅
```

View Serializability

View serializability is a weaker (less strict) form than conflict serializability. A schedule S is view-serializable if it is view-equivalent to some serial schedule. Two schedules are view-equivalent if:

1. Each transaction reads the same initial values
2. Each read-write pair sees the same write
3. The final write to each data item is from the same transaction

Every conflict-serializable schedule is view-serializable. The converse is NOT true. View serializability allows **blind writes** (writes without a preceding read). Checking

view serializability is NP-hard — which is why databases use conflict serializability (checked via the precedence graph) in practice.

5.4 Concurrency Protocols

Two-Phase Locking (2PL)

The most widely used protocol. Every transaction follows two phases:
Phase 1 — Growing phase: The transaction acquires locks but does NOT release any. It can lock shared (S) locks for reads and exclusive (X) locks for writes.

Phase 2 — Shrinking phase: The transaction releases locks but does NOT acquire any new ones.

Why 2PL guarantees serializability: The lock point (where the growing phase ends and shrinking begins) defines a total ordering of transactions. Transactions are effectively serialized at their lock points.

The problem — Cascading Rollbacks: If T1 modifies X, releases the X lock (in shrinking phase), and T2 reads X (dirty read), then T1 must abort — but now T2 must also abort because it read dirty data. If T3 read from T2... the cascade continues.

Strict 2PL: Exclusive locks are held until COMMIT or ABORT. This prevents dirty reads and cascading rollbacks. All reads can still happen. Standard in most DBs.

Rigorous 2PL: ALL locks (both S and X) are held until COMMIT/ABORT. Prevents the shrinking phase from releasing any lock. This simplifies recovery but reduces concurrency.

Conservative (Static) 2PL: All locks are acquired BEFORE the transaction begins. This prevents deadlocks (you either get all your locks or none), but it requires knowing the full data access pattern upfront — often impractical.

Lock compatibility matrix:

	S-lock	X-lock
S-lock	Compatible ✅	Incompatible ❌
X-lock	Incompatible ❌	Incompatible ❌

Timestamp Ordering (TO)

Every transaction T gets a unique timestamp TS(T) when it starts. Every data item X has two timestamps:

- `read_TS(X)` — timestamp of the last transaction that read X
- `write_TS(X)` — timestamp of the last transaction that wrote X

Rules:

- **For T reads X:** If TS(T) < write_TS(X), T is trying to read a value already overwritten by a newer transaction — ABORT T. Otherwise, allow and update read_TS(X) = max(read_TS(X), TS(T)).
- **For T writes X:** If TS(T) < read_TS(X), a newer transaction already read X and expected T not to write it — ABORT T. If TS(T) < write_TS(X), a newer transaction already wrote X — skip this write (**Thomas Write Rule**). Otherwise, write and update write_TS(X) = TS(T).

Thomas Write Rule: Instead of aborting when TS(T) < write_TS(X), we simply ignore T's write. T's write is "obsolete" — a newer transaction has already written a more recent value. This increases concurrency by reducing unnecessary aborts.

5.5 Transaction Isolation Levels

This is one of the most practically important topics in database interviews and real system design. The SQL standard defines four isolation levels that make different trade-offs between data correctness and concurrency performance.

The Three Concurrency Anomalies

Dirty Read: Transaction T1 reads a value written by T2, but T2 has NOT yet committed. If T2 aborts, T1 has read data that never officially existed.

MediBook example: T2 is updating a patient's consultation fee to ₹0 (bug). T1 reads this ₹0 value and skips billing. T2 aborts and the fee reverts to ₹800. The appointment was given for free because of a dirty read.

Non-Repeatable Read: Transaction T1 reads a value at time t1. T2 updates and commits that value. T1 reads the same value again at time t2 (within the same transaction) and gets a different result.

MediBook example: T1 is generating an invoice and reads the consultation fee as ₹800. T2 (admin) updates the fee to ₹1200 and commits. T1 reads the fee again to print on the invoice — now it says ₹1200. The same transaction read different values for the same row.

Phantom Read: T1 executes a range query and gets a set of rows. T2 inserts new rows matching T1's criteria. T1 re-executes the same query and gets a different (larger) set of rows — "phantom" rows appeared.

MediBook example: T1 counts available appointment slots for Dr. Kapoor on Monday: gets 3. T2 bulk-books 2 of those slots and commits. T1 re-queries and gets 1. T1 is confused — it expected 3 slots to still be available.

Isolation Levels and Their Guarantees

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible ❌	Possible ❌	Possible ❌
Read Committed	Not Possible ✅	Possible ❌	Possible ❌
Repeatable Read	Not Possible ✅	Not Possible ✅	Possible ❌
Serializable	Not Possible ✅	Not Possible ✅	Not Possible ✅

Detailed Explanation of Each Level

Read Uncommitted — The lowest, most dangerous level. A transaction can read data that other in-progress transactions have written but not yet committed. This is almost never appropriate for real applications. Use case: approximate analytics where a few percent error is acceptable (e.g., counting approximate page views).

Read Committed — The default isolation level in PostgreSQL and Oracle. A transaction only sees data that has been committed by other transactions. Each SQL statement within the transaction gets a fresh snapshot — so two SELECT statements within the same transaction CAN see different data if another transaction commits between them. This prevents dirty reads but allows non-repeatable reads.

```sql
-- In PostgreSQL, default is READ COMMITTED:
BEGIN;
SELECT consultation_fee FROM doctors WHERE doctor_id = 1; -- Returns 800
-- Another session commits an update: fee now = 1200
SELECT consultation_fee FROM doctors WHERE doctor_id = 1; -- Returns 1200 (non-re
COMMIT;
```

Repeatable Read — Each transaction sees a consistent snapshot of the database as it existed when the transaction STARTED. All reads within the transaction see the same data regardless of commits by other transactions. This prevents dirty reads and non-repeatable reads. PostgreSQL's implementation also prevents phantom reads (making it stronger than the SQL standard requires for this level), using MVCC.

```sql
```

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT consultation_fee FROM doctors WHERE doctor_id = 1; -- Returns 800
-- Another session commits: fee = 1200
SELECT consultation_fee FROM doctors WHERE doctor_id = 1; -- Still returns 800 (
COMMIT;
```

Serializable — The strictest level. Transactions execute as if they ran one after another in some serial order. PostgreSQL implements this via Serializable Snapshot Isolation (SSI), which detects and aborts transactions that would have produced non-serializable outcomes. Applications must handle serialization errors (error code 40001) with retry logic.

sql

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- All reads are perfectly isolated; if another transaction creates a conflict,
-- one of the transactions will be aborted with a serialization error
COMMIT;
```

Setting Isolation Levels in PostgreSQL

sql

```
-- For a single transaction:
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED; -- default
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Change default for the session:
SET default_transaction_isolation = 'repeatable read';

-- Check current isolation level:
SHOW transaction_isolation;
```

5.6 Deadlock Handling & MVCC

Deadlocks

A deadlock occurs when two or more transactions are each waiting for a lock held by another, creating a circular wait that no one can break.

Classic example:

T1 waits for T2, T2 waits for T1. Neither can proceed. The system is frozen.

Detection: The DBMS maintains a **wait-for graph**: directed edges from waiting transaction to the transaction holding the needed lock. A cycle in this graph indicates a deadlock. PostgreSQL runs deadlock detection periodically (every `deadlock_timeout` milliseconds, default 1s).

Resolution: The DBMS chooses a **victim** transaction to abort (usually the one with the least work done, or the youngest). The victim is rolled back, releasing its locks, which allows other transactions to proceed. The application should be designed to retry the aborted transaction.

```sql
-- PostgreSQL logs deadlocks; to see them:
-- Look in PostgreSQL log for: "ERROR: deadlock detected"
-- Application code should catch error code 40P01 and retry:
-- try { COMMIT } catch (deadlock_error) { retry_transaction() }
```

Prevention strategies:

- Always acquire locks in the SAME ORDER across all transactions (eliminates circular waits)
- Use Conservative 2PL (acquire all locks upfront)
- Use row-level locking instead of table-level locking (reduces lock scope)
- Keep transactions short (reduce the time window for conflicts)

Multi-Version Concurrency Control (MVCC)

MVCC is the revolutionary technique used by PostgreSQL, Oracle, and MySQL InnoDB that eliminates the read-write lock conflict — readers never block writers, writers never block readers.

The Core Idea: Instead of updating data in-place and locking it, the DBMS keeps **multiple versions** of each row. Writers create a new version without overwriting the old one. Readers access the version that was current at their transaction's start time.

How PostgreSQL implements MVCC:

Every row (heap tuple) in PostgreSQL has two hidden system columns:

- `xmin` — the transaction ID that CREATED this row version

- `xmax` — the transaction ID that DELETED or UPDATED this row version (0 if still current)

```sql
-- You can see these hidden columns:
SELECT xmin, xmax, patient_id, first_name FROM patients LIMIT 5;
```

Read scenario:

- T1 starts at timestamp t1 (transaction ID = 100)
- T2 updates patient P1's phone number at timestamp t2 (transaction ID = 101)
- T1 then reads P1

In MVCC, T2's update creates a NEW row version for P1 with `xmin=101`. The OLD version has `xmax=101` (marking it as deleted by T2). T1, which started before T2 (transaction ID 100 < 101), looks for row versions where `xmin <= 100` and `xmax = 0 OR xmax > 100`. T1 finds the OLD version and reads it — WITHOUT any locking needed. T2 and T1 ran concurrently without blocking each other.

Vacuum: The old row versions (dead tuples) accumulate over time. PostgreSQL's `VACUUM` process periodically cleans these up. `AUTOVACUUM` handles this automatically. This is why PostgreSQL tables can grow and need occasional maintenance.

Benefits:

- Read queries never wait for write transactions
- Write transactions never wait for read transactions
- Near-elimination of read/write contention (the most common source of lock waits)
- Consistent snapshots across a transaction without locking every row

Trade-off: Table bloat from dead tuples. The need for VACUUM. Slightly higher write overhead (creating new versions instead of in-place updates).

5.7 Recovery Systems

Write-Ahead Logging (WAL)

WAL is the foundation of database durability. The rule is simple: **before any data page is modified on disk, the change must first be written to the WAL log.**

```
Transaction flow:
1. T1 begins
2. T1 reads data from disk into buffer pool (RAM)
3. T1 modifies the in-memory copy
4. BEFORE writing to disk: the change record is appended to WAL (sequential
write)
5. T1 commits → WAL is flushed to disk (fsync)
6. System replies "committed" to the client
7. (Later) Modified data pages are written from buffer pool to disk
(background)
```

The WAL contains:

- `<BEGIN T1>` — transaction start marker
- `<T1, X, old_value, new_value>` — before and after values for each change
- `<COMMIT T1>` — commit marker

Crash recovery using WAL (ARIES algorithm concepts):

1. **Analysis phase:** Read WAL to determine which transactions committed and which were in-progress at crash time
2. **Redo phase:** Replay ALL changes from WAL (even uncommitted ones) to bring data files to the exact state at crash time
3. **Undo phase:** Undo (rollback) all changes from transactions that were NOT committed at crash time

Checkpoints

If the WAL were never truncated, recovery would require replaying the entire WAL from the beginning — potentially days or weeks of log. **Checkpoints** solve this by periodically creating a consistent disk snapshot.

At a checkpoint:

1. All dirty (modified) buffer pages are flushed to disk
2. A `<CHECKPOINT>` record is written to the WAL
3. WAL segments before the checkpoint can be safely archived or deleted

Recovery now only needs to replay from the last checkpoint, not from the beginning. PostgreSQL runs checkpoints automatically (controlled by `checkpoint_timeout` and `max_wal_size`).

```sql
```

```
-- Manually force a checkpoint:
CHECKPOINT;

-- View checkpoint activity:
SELECT * FROM pg_stat_bgwriter;
```

Shadow Paging

An alternative recovery technique (not used by PostgreSQL but important conceptually). The database maintains two page tables: the **current page table** (active) and a **shadow page table** (backup copy from the last consistent state).

When a transaction modifies data, it writes to new pages on disk and updates the CURRENT page table to point to the new pages. The SHADOW page table still points to the old pages. If the transaction aborts, discard the new pages and restore the current table to match the shadow. If it commits, update the shadow table to match the current table (atomic pointer swap).

Advantages: Simple recovery (no undo/redo log needed). **Disadvantages:** External fragmentation (modified pages are spread across disk), expensive commits (entire page table must be written), poor concurrency. This is why most systems prefer WAL-based recovery.

---

6. Modern System Design Concepts

6.1 SQL vs. NoSQL

The SQL vs. NoSQL debate is not about which is "better" — it's about choosing the right tool for the right problem. Understanding when to use each is a critical senior engineering skill.

SQL (Relational) Databases

SQL databases organize data into structured tables with predefined schemas, enforce relationships via foreign keys, and guarantee ACID transactions. They use structured query language for all operations.

Best suited for:

- Data with well-defined, stable structure
- Applications requiring complex queries joining multiple related entities
- Systems where data integrity and consistency are paramount (banking, medical records, billing)
- OLTP workloads with many short, concurrent read-write transactions

Examples: PostgreSQL, MySQL, Oracle, SQL Server, SQLite

NoSQL Databases — Four Main Types

1. Key-Value Stores (Redis, DynamoDB, Riak)

Imagine a massive dictionary: you store a value under a key, and retrieve it by that key. No schema, no relationships, no complex queries.

In MediBook, Redis is used as:

- **Session cache:** Store user session data `{session_id → user_object}` with a TTL (time-to-live). Blazing fast retrieval (< 1ms) from RAM.
- **Rate limiting:** `{user_id:endpoint → request_count}` to prevent API abuse.
- **Appointment availability cache:** Pre-compute available slots and cache them: `{doctor_id:date → [available_times]}`.

Use case fit: When you need O(1) lookups, caching, pub/sub messaging, leaderboards, and session management.

2. Document Stores (MongoDB, CouchDB, Firestore)

Documents are semi-structured data (usually JSON or BSON) grouped into collections. Each document can have a different structure (schema-flexible). Documents can

embed related data hierarchically.

```json
// A patient document in MongoDB MediBook:
{
  "_id": "P123",
  "name": { "first": "Riya", "last": "Menon" },
  "contact": { "email": "riya@example.com", "phone": "9876543210" },
  "appointments": [
    { "date": "2024-01-15", "doctor": "Dr. Sharma", "status": "completed" },
    { "date": "2024-03-20", "doctor": "Dr. Patel", "status": "scheduled" }
  ],
  "medical_history": ["hypertension", "diabetes_type_2"]
}
```

Use case fit: Content management systems, product catalogs, user profiles, event logs. Excellent when the schema evolves rapidly or each record genuinely has different attributes.

Weakness: No true joins — embedding creates redundancy. If Dr. Sharma's name changes, it must be updated in every patient's appointment array. This is the NoSQL version of a normalization problem.

3. Columnar Stores (Apache Cassandra, HBase, Google Bigtable, AWS Redshift)

Data is organized by COLUMNS rather than rows. Instead of storing all columns of row 1 together, row 2 together, etc., a columnar store stores all values of column 1 together, all values of column 2 together.

```
Row-oriented (PostgreSQL):
Row 1: [patient_id=1, name="Riya", city="Mumbai", age=28]
Row 2: [patient_id=2, name="Amit", city="Delhi", age=35]

Column-oriented (Redshift/Parquet):
patient_id column: [1, 2, 3, 4, ...]
name column:       ["Riya", "Amit", "Priya", "Suresh", ...]
city column:       ["Mumbai", "Delhi", "Bangalore", "Chennai", ...]
age column:        [28, 35, 29, 42, ...]
```

Why this matters for analytics: The query "average age of all patients" only needs the `age` column. In a columnar store, you read ONLY the age column (a tiny fraction of the data). In a row store, you'd read every row to extract the age field from each. For analytical queries on billions of rows, this is a 10-100x speedup.

Bonus: Columnar data compresses dramatically better (a column full of the same data type and similar values = much better compression ratio).

Use case fit: Data warehouses, analytics platforms, time-series data, reporting. NOT good for OLTP (updating a single row requires touching many column files).

Cassandra specifically: Wide-column store optimized for massive write throughput across many nodes. Used for IoT sensor data, activity feeds, messaging platforms.

4. Graph Databases (Neo4j, Amazon Neptune, JanusGraph)

Data is modeled as **nodes** (entities) and **edges** (relationships). Graph traversal is the primary query pattern.

In a TalentCall (video interview platform) context, a graph DB is excellent for modeling:

- Candidates and their connections (mutual referrals, shared employers)
- "Find all candidates recommended by current employees within 3 degrees of connection"
- Skills relationships: Skill A is prerequisite for Skill B is related to Skill C

```cypher
// Neo4j Cypher query for "candidates connected to our employees":
MATCH (e:Employee)-[:REFERRED]->(c:Candidate)
WHERE e.company = 'TechCorp'
RETURN c.name, c.skills
```

In a relational database, this query would require recursive CTEs or multiple self-joins and would be extremely expensive for large graphs.

SQL vs. NoSQL Decision Matrix

Criterion	SQL	NoSQL
Data structure	Fixed schema, tabular	Flexible, varies by type
ACID transactions	Full support	Eventual consistency (mostly)
Complex queries	Excellent (JOINs, aggregations)	Limited (varies by type)
Horizontal scaling	Harder	Designed for it
Read/write speed	Good	Often faster (no joins)
Schema flexibility	Low	High
Maturity/tooling	40+ years	15-20 years

Criterion	SQL	NoSQL
Best for	OLTP, consistency-critical	High scale, schema-flexible data

6.2 CAP Theorem

The CAP Theorem (Brewer's Theorem, 2000) states that a distributed data system can only guarantee **at most two of three** properties simultaneously:

Consistency (C): Every read receives the most recent write, or returns an error. All nodes in the cluster see the same data at the same time.

Availability (A): Every request receives a response (not an error) — though the response might not contain the most recent data.

Partition Tolerance (P): The system continues to operate even if network partitions (communication failures between nodes) occur.

The key insight: In a real distributed system, network partitions are inevitable (hardware fails, network cables break, data centers lose connectivity). You CANNOT eliminate P in practice. So the real choice in a distributed system is:

CP (Consistency + Partition Tolerance): When a partition occurs, the system becomes unavailable rather than returning stale data. Used when consistency is non-negotiable.

- Examples: HBase, Zookeeper, MongoDB (with certain configurations), PostgreSQL in synchronous replication
- MediBook context: appointment booking system must be CP — double-booking is unacceptable

AP (Availability + Partition Tolerance): When a partition occurs, the system keeps responding but might return stale data. Eventually, when the partition heals, data converges (**eventual consistency**).

- Examples: Cassandra, DynamoDB (default), CouchDB
- MediBook context: the patient notification system (SMS/email queue) can be AP — a slight delay or duplicate notification is acceptable, but going down is not

CA (Consistency + Availability): Only possible in a single-node system with no network partitions. This category doesn't really exist for truly distributed systems. Traditional single-server RDBMS are CA.

PACELC Theorem (extension): Even when the system is running normally (no partition), there's a trade-off between Latency (L) and Consistency (C). PACELC more accurately captures the full design space.

6.3 Database Scaling

Vertical Scaling (Scale Up)

The simplest approach: use a more powerful machine. More RAM, faster CPUs, faster SSDs.

Pros: No application changes, no complexity added. **Cons:** Physical limits exist (you can't scale infinitely). Expensive for high-end hardware. Single point of failure.

Horizontal Scaling (Scale Out)

Distribute the database across many commodity machines. There are three main patterns:

Replication — Master-Slave (Primary-Replica) Architecture

One **primary** (master) node handles all WRITES. One or more **replica** (slave) nodes receive copies of all writes and handle READ traffic.

MediBook use case: The appointment booking (write) goes to the primary. Read queries — doctor search, patient history viewing, report generation — go to replicas. If 90% of traffic is reads, adding 3 replicas effectively 4x's read capacity.

Replication lag: There's a small delay between a write on primary and it appearing on replicas. In asynchronous replication, a patient might book an appointment and then immediately refresh the page (hitting a replica) and not see their appointment yet. This is eventual consistency within a replication setup.

Synchronous replication: The primary waits for at least one replica to confirm the write before acknowledging to the client. Eliminates replication lag but adds latency to every write.

```sql
```

```
-- PostgreSQL replication setup (primary postgresql.conf):
wal_level = replica
max_wal_senders = 3
synchronous_standby_names = 'replica1'  -- for synchronous replication
```

Sharding (Horizontal Partitioning)

Sharding distributes DIFFERENT data across multiple nodes. Each node (shard) holds a subset of the rows. Unlike replication (where every node has all data), in sharding each piece of data lives on exactly one shard.

Example: Shard the `appointments` table by `patient_id`:

- Shard 1: patients with patient_id 1–1,000,000
- Shard 2: patients with patient_id 1,000,001–2,000,000
- Shard 3: patients with patient_id 2,000,001–3,000,000

Sharding strategies:

- **Range-based:** Shard by value range (dates, ID ranges). Simple, supports range queries on the shard key. Risk: hot shards if recent dates are all on one shard.
- **Hash-based:** Hash the shard key and assign to a shard. Uniform distribution. Breaks range queries (data for the same date range is spread across all shards).
- **Geographic/Directory-based:** Route based on a lookup table or geography (India patients → shard1, US patients → shard2). Flexible but requires a routing layer.

Cross-shard joins — the big problem: If a query needs data from multiple shards, the application (or a proxy layer) must query multiple shards and merge results. This is complex, slow, and removes the JOIN simplicity of SQL. This is why sharding is a last resort.

Partitioning (Within a Single Node)

Partitioning divides a large table into smaller pieces **within the same database**, for manageability and query performance. Unlike sharding, all partitions are on one server (though partitions can be moved to different tablespaces).

sql

```
-- Partition appointments by year (Range Partitioning in PostgreSQL):
CREATE TABLE appointments (
    appointment_id SERIAL,
    patient_id INT,
    appointment_date DATE NOT NULL,
    status VARCHAR(20)
) PARTITION BY RANGE (appointment_date);

CREATE TABLE appointments_2023 PARTITION OF appointments
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE appointments_2024 PARTITION OF appointments
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE appointments_2025 PARTITION OF appointments
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

-- PostgreSQL automatically routes INSERTs and scans only relevant partitions:
SELECT * FROM appointments WHERE appointment_date = '2024-06-15';
-- Postgres only scans appointments_2024 — partition pruning!
```

6.4 Connection Pooling & Query Caching

Connection Pooling

Opening a new database connection is expensive (TCP handshake, authentication, memory allocation). In a web application with 1000 concurrent users, creating 1000 database connections simultaneously would exhaust resources.

A **connection pool** maintains a pool of pre-opened connections that are reused. A client borrows a connection, uses it, and returns it to the pool.

PgBouncer is the most popular PostgreSQL connection pooler:

```
Application (1000 requests/s) → PgBouncer (20 connections) → PostgreSQL
```

PgBouncer's three modes:

- **Session mode:** One server connection per client session. Minimal benefit but highest compatibility.
- **Transaction mode:** Connection is returned to pool after each transaction completes. Most common. Reduces connections dramatically.
- **Statement mode:** Connection returned after each statement. Highest reuse. Incompatible with multi-statement transactions.

Query Result Caching

For expensive read queries that run frequently and return the same result, caching the result in Redis/Memcached avoids hitting the database at all.

```
Request → Check Redis cache → Cache hit → Return cached result (< 1ms)
                            → Cache miss → Query PostgreSQL → Store in Redis →
Return result
```

MediBook example: "List all active doctors by city" — this changes rarely but is queried on every doctor search. Cache the result with a 5-minute TTL.

python

```python
def get_doctors_by_city(city):
    cache_key = f"doctors:city:{city}"
    cached = redis.get(cache_key)
    if cached:
        return json.loads(cached)
    doctors = db.query("SELECT * FROM doctors WHERE city = %s", city)
    redis.setex(cache_key, 300, json.dumps(doctors))  # 300s TTL
    return doctors
```

Cache invalidation (one of the hardest problems in CS): When a doctor's data changes, you must invalidate the relevant cache entries. Strategies: TTL-based expiry, explicit invalidation on write, event-driven invalidation via database triggers.

---

7. SQL Query Mastery — 50 Queries

> **Schema Setup Reference:** The following queries use the MediBook schema: `patients`, `doctors`, `appointments`, `hospitals`, `specializations`, `doctor_specializations`, `prescriptions`, `doctor_phones`.

7.1 DDL, DML, DCL, TCL Basics (Q1–Q5)

Q1 — DDL: Create and Alter Table with Constraints

sql

```sql
-- Create the lab_tests table with proper constraints
CREATE TABLE lab_tests (
    test_id          SERIAL PRIMARY KEY,
    appointment_id   INT NOT NULL REFERENCES appointments(appointment_id) ON DELE
    test_name        VARCHAR(200) NOT NULL,
    result           TEXT,
    result_status    VARCHAR(20) DEFAULT 'pending'
                       CHECK (result_status IN ('pending','normal','abnormal','cr
    tested_on        TIMESTAMPTZ,
    lab_name         VARCHAR(150),
    created_at       TIMESTAMPTZ DEFAULT NOW()
);

-- Add a new column for lab technician name:
ALTER TABLE lab_tests ADD COLUMN technician_name VARCHAR(100);

-- Add a check constraint after creation:
ALTER TABLE lab_tests ADD CONSTRAINT chk_tested_after_created
    CHECK (tested_on >= created_at OR tested_on IS NULL);

-- Add an index on result_status for fast filtering:
CREATE INDEX idx_lab_test_status ON lab_tests(result_status);

-- Drop a column:
ALTER TABLE lab_tests DROP COLUMN IF EXISTS lab_name;
```

Q2 — DML: INSERT with Conflict Handling (UPSERT)

sql

```sql
-- Insert a doctor, update if doctor_id already exists (UPSERT):
INSERT INTO doctors (doctor_id, first_name, last_name, email, consultation_fee, h
VALUES (101, 'Priya', 'Sharma', 'priya.sharma@medibook.com', 900.00, 5)
ON CONFLICT (email)
DO UPDATE SET
    consultation_fee = EXCLUDED.consultation_fee,
    first_name = EXCLUDED.first_name,
    last_name = EXCLUDED.last_name;

-- Insert multiple patients at once:
INSERT INTO patients (first_name, last_name, date_of_birth, gender, phone, email,
VALUES
    ('Riya',   'Menon',   '1996-04-12', 'F', '9876543210', 'riya@ex.com',   'Mumba
    ('Amit',   'Shah',    '1988-09-25', 'M', '9988776655', 'amit@ex.com',   'Delhi
    ('Sunita', 'Reddy',   '1975-12-01', 'F', '9123456789', 'sun@ex.com',    'Hyder
    ('Karan',  'Malhotra','2001-03-18', 'M', '9001234567', 'karan@ex.com',  'Pune'
```

Q3 — DML: UPDATE with Joins and Subqueries

```sql
-- Give a 10% fee increase to all doctors at Apollo Hospital who have
-- more than 10 completed appointments:
UPDATE doctors d
SET consultation_fee = consultation_fee * 1.10
WHERE d.hospital_id = (SELECT hospital_id FROM hospitals WHERE hospital_name = 'A
  AND d.doctor_id IN (
    SELECT doctor_id
    FROM appointments
    WHERE status = 'completed'
    GROUP BY doctor_id
    HAVING COUNT(*) > 10
);
```

Q4 — DCL: Granting and Revoking Privileges

```sql
```

```sql
-- Create a read-only role for BI analysts:
CREATE ROLE bi_analyst;
GRANT CONNECT ON DATABASE medibook TO bi_analyst;
GRANT USAGE ON SCHEMA public TO bi_analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO bi_analyst;
-- Ensure future tables are also readable:
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO bi_analyst;

-- Create a receptionist user who can manage appointments:
CREATE ROLE receptionist LOGIN PASSWORD 'securepassword123';
GRANT SELECT, INSERT, UPDATE ON appointments TO receptionist;
GRANT SELECT ON patients TO receptionist;
GRANT SELECT ON doctors TO receptionist;
-- But NOT delete (accidental deletion prevention):
REVOKE DELETE ON appointments FROM receptionist;
```

Q5 — TCL: Transaction with Savepoints

sql

```sql
-- Booking an appointment with a savepoint to handle partial failures:
BEGIN;

-- Step 1: Create the appointment
INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_t
VALUES (42, 7, '2025-02-10', '10:00', 'scheduled', 'Annual checkup')
RETURNING appointment_id INTO appointment_id_var;

SAVEPOINT after_appointment;

-- Step 2: Attempt to charge the wallet (might fail if insufficient balance)
UPDATE patient_wallets
SET balance = balance - (SELECT consultation_fee FROM doctors WHERE doctor_id = 7
WHERE patient_id = 42
  AND balance >= (SELECT consultation_fee FROM doctors WHERE doctor_id = 7);

-- If no row updated (insufficient balance), rollback to savepoint
-- (in real code, check GET DIAGNOSTICS or ROW_COUNT)
-- ROLLBACK TO SAVEPOINT after_appointment;

COMMIT;
```

7.2 Advanced Joins & Set Operations (Q6–Q15)

Q6 — INNER JOIN: Appointment Details with Doctor and Patient Info

sql

```sql
SELECT
    a.appointment_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    h.hospital_name,
    a.appointment_date,
    a.appointment_time,
    a.status,
    d.consultation_fee
FROM appointments a
INNER JOIN patients p ON p.patient_id = a.patient_id
INNER JOIN doctors d ON d.doctor_id = a.doctor_id
INNER JOIN hospitals h ON h.hospital_id = d.hospital_id
WHERE a.appointment_date >= CURRENT_DATE
ORDER BY a.appointment_date, a.appointment_time;
```

Q7 — LEFT JOIN: All Patients Including Those With No Appointments

sql

```sql
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    p.email,
    COUNT(a.appointment_id) AS total_appointments,
    MAX(a.appointment_date) AS last_visit_date
FROM patients p
LEFT JOIN appointments a ON a.patient_id = p.patient_id
GROUP BY p.patient_id, p.first_name, p.last_name, p.email
ORDER BY total_appointments DESC;
```

Q8 — SELF JOIN: Find Patients Who Share the Same Doctor

sql

```sql
SELECT DISTINCT
    p1.patient_id   AS patient1_id,
    CONCAT(p1.first_name, ' ', p1.last_name) AS patient1_name,
    p2.patient_id   AS patient2_id,
    CONCAT(p2.first_name, ' ', p2.last_name) AS patient2_name,
    CONCAT(d.first_name, ' ', d.last_name)   AS shared_doctor
FROM appointments a1
JOIN appointments a2 ON a1.doctor_id = a2.doctor_id
                    AND a1.patient_id < a2.patient_id  -- avoid duplicates (A,B)
JOIN patients p1 ON p1.patient_id = a1.patient_id
JOIN patients p2 ON p2.patient_id = a2.patient_id
JOIN doctors d ON d.doctor_id = a1.doctor_id
ORDER BY shared_doctor, patient1_name;
```

Q9 — FULL OUTER JOIN: Doctors and Patients in Same City (Including Unmatched)

sql

```sql
-- Find all cities: doctors' cities vs patients' cities, including cities with or
SELECT
    COALESCE(d_city.city, p_city.city) AS city,
    COALESCE(d_city.doctor_count, 0) AS doctors_in_city,
    COALESCE(p_city.patient_count, 0) AS patients_in_city
FROM (
    SELECT h.city, COUNT(DISTINCT d.doctor_id) AS doctor_count
    FROM doctors d JOIN hospitals h ON h.hospital_id = d.hospital_id
    GROUP BY h.city
) d_city
FULL OUTER JOIN (
    SELECT city, COUNT(*) AS patient_count
    FROM patients
    GROUP BY city
) p_city ON d_city.city = p_city.city
ORDER BY city;
```

Q10 — CROSS JOIN: Generate All Possible Doctor-Patient Combinations for a Date

sql

```sql
-- Find all (doctor, patient) pairs that do NOT yet have an appointment on a spec
-- Useful for a scheduling assistant to suggest bookings.
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name
FROM doctors d
CROSS JOIN patients p
WHERE NOT EXISTS (
    SELECT 1 FROM appointments a
    WHERE a.doctor_id = d.doctor_id
      AND a.patient_id = p.patient_id
      AND a.appointment_date = '2025-03-15'
)
ORDER BY doctor_name, patient_name;
```

Q11 — JOIN with Aggregation: Top 5 Doctors by Revenue

sql

```sql
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    h.hospital_name,
    COUNT(a.appointment_id)            AS completed_appointments,
    SUM(d.consultation_fee)            AS total_revenue,
    ROUND(AVG(d.consultation_fee), 2)  AS avg_fee
FROM doctors d
JOIN hospitals h ON h.hospital_id = d.hospital_id
JOIN appointments a ON a.doctor_id = d.doctor_id
WHERE a.status = 'completed'
GROUP BY d.doctor_id, d.first_name, d.last_name, h.hospital_name, d.consultation_
ORDER BY total_revenue DESC
LIMIT 5;
```

Q12 — Multi-Table JOIN with Specialization Filter

sql

```sql
-- Find all cardiologists in Mumbai with their appointment counts this year:
SELECT
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    d.consultation_fee,
    h.hospital_name,
    COUNT(a.appointment_id) AS appointments_this_year
FROM doctors d
JOIN hospitals h ON h.hospital_id = d.hospital_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
LEFT JOIN appointments a ON a.doctor_id = d.doctor_id
    AND EXTRACT(YEAR FROM a.appointment_date) = EXTRACT(YEAR FROM CURRENT_DATE)
WHERE s.name = 'Cardiology'
  AND h.city = 'Mumbai'
GROUP BY d.doctor_id, d.first_name, d.last_name, d.consultation_fee, h.hospital_
ORDER BY appointments_this_year DESC;
```

Q13 — UNION: All People (Patients + Doctors) Who Live in Delhi

sql

```sql
SELECT 'Patient' AS role, patient_id AS id, first_name, last_name, email, city
FROM patients
WHERE city = 'Delhi'
UNION
SELECT 'Doctor' AS role, d.doctor_id, d.first_name, d.last_name, d.email, h.city
FROM doctors d
JOIN hospitals h ON h.hospital_id = d.hospital_id
WHERE h.city = 'Delhi'
ORDER BY role, last_name;
```

Q14 — INTERSECT: Patients Who Have Seen Both a Cardiologist and a Neurologist

sql

```sql
SELECT patient_id FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
WHERE s.name = 'Cardiology'
INTERSECT
SELECT patient_id FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
WHERE s.name = 'Neurology';
```

Q15 — EXCEPT: Doctors Who Have Appointments But No Completed Ones

sql

```sql
-- Find doctors who have some appointments but none are 'completed' —
-- they may have a high cancellation or no-show rate
SELECT DISTINCT doctor_id
FROM appointments
WHERE status IN ('scheduled', 'cancelled', 'no_show')
EXCEPT
SELECT DISTINCT doctor_id
FROM appointments
WHERE status = 'completed';
```

7.3 Subqueries — Nested & Correlated (Q16–Q25)

Q16 — Scalar Subquery: Appointments Above Average Fee

sql

```sql
SELECT
    a.appointment_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    d.consultation_fee,
    (SELECT ROUND(AVG(consultation_fee), 2) FROM doctors) AS avg_fee_all_doctors
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
WHERE d.consultation_fee > (SELECT AVG(consultation_fee) FROM doctors)
  AND a.status = 'completed'
ORDER BY d.consultation_fee DESC;
```

Q17 — IN Subquery: Patients Who Have Critical Lab Results

sql

```sql
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    p.email,
    p.phone
FROM patients p
WHERE p.patient_id IN (
    SELECT DISTINCT a.patient_id
    FROM appointments a
    JOIN lab_tests lt ON lt.appointment_id = a.appointment_id
    WHERE lt.result_status = 'critical'
)
ORDER BY p.last_name;
```

Q18 — NOT IN Subquery: Doctors With No Appointments This Month

sql

```sql
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    h.hospital_name,
    d.consultation_fee
FROM doctors d
JOIN hospitals h ON h.hospital_id = d.hospital_id
WHERE d.doctor_id NOT IN (
    SELECT DISTINCT doctor_id
    FROM appointments
    WHERE DATE_TRUNC('month', appointment_date) = DATE_TRUNC('month', CURRENT_DAT
)
ORDER BY doctor_name;
```

Q19 — EXISTS Subquery: Patients With At Least One Prescription

sql

```sql
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    p.date_of_birth,
    EXTRACT(YEAR FROM AGE(p.date_of_birth)) AS age
FROM patients p
WHERE EXISTS (
    SELECT 1
    FROM appointments a
    JOIN prescriptions pr ON pr.appointment_id = a.appointment_id
    WHERE a.patient_id = p.patient_id
)
ORDER BY age DESC;
```

Q20 — NOT EXISTS Subquery: Specializations With No Doctors

sql

```sql
SELECT s.specialization_id, s.name, s.description
FROM specializations s
WHERE NOT EXISTS (
    SELECT 1
    FROM doctor_specializations ds
    WHERE ds.specialization_id = s.specialization_id
)
ORDER BY s.name;
```

Q21 — Correlated Subquery: Each Doctor's Most Recent Appointment

sql

```sql
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    (
        SELECT a.appointment_date
        FROM appointments a
        WHERE a.doctor_id = d.doctor_id
        ORDER BY a.appointment_date DESC
        LIMIT 1
    ) AS last_appointment_date,
    (
        SELECT a.status
        FROM appointments a
        WHERE a.doctor_id = d.doctor_id
        ORDER BY a.appointment_date DESC
        LIMIT 1
    ) AS last_appointment_status
FROM doctors d
ORDER BY last_appointment_date DESC NULLS LAST;
```

Q22 — Correlated Subquery: Patients Who Spent More Than Average Among Their City

sql

```sql
-- For each patient, compare their total spending to the average spending
-- of patients in the same city:
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    p.city,
    SUM(d.consultation_fee) AS total_spent
FROM patients p
JOIN appointments a ON a.patient_id = p.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
WHERE a.status = 'completed'
GROUP BY p.patient_id, p.first_name, p.last_name, p.city
HAVING SUM(d.consultation_fee) > (
    SELECT AVG(city_spending.total)
    FROM (
        SELECT p2.patient_id, SUM(d2.consultation_fee) AS total
        FROM patients p2
        JOIN appointments a2 ON a2.patient_id = p2.patient_id
        JOIN doctors d2 ON d2.doctor_id = a2.doctor_id
        WHERE a2.status = 'completed' AND p2.city = p.city
        GROUP BY p2.patient_id
    ) city_spending
)
ORDER BY total_spent DESC;
```

Q23 — Subquery in FROM Clause (Derived Table): Monthly Appointment Counts

sql

```sql
SELECT
    monthly.year,
    monthly.month,
    monthly.total_appointments,
    monthly.completed,
    monthly.cancelled,
    ROUND(100.0 * monthly.completed / NULLIF(monthly.total_appointments, 0), 2) A
FROM (
    SELECT
        EXTRACT(YEAR FROM appointment_date)  AS year,
        EXTRACT(MONTH FROM appointment_date) AS month,
        COUNT(*)                              AS total_appointments,
        COUNT(*) FILTER (WHERE status = 'completed')  AS completed,
        COUNT(*) FILTER (WHERE status = 'cancelled')  AS cancelled
    FROM appointments
    GROUP BY year, month
) monthly
ORDER BY year, month;
```

Q24 — CTE (WITH clause): Recursive CTE for Hierarchical Data

sql

```sql
-- Using CTE to find doctor referral chains
-- (Assuming a doctor_referrals table: referrer_id → referred_id)
WITH RECURSIVE referral_chain AS (
    -- Base case: Start with a specific doctor
    SELECT
        doctor_id AS referred_doctor,
        doctor_id AS referrer,
        0 AS depth,
        ARRAY[doctor_id] AS path
    FROM doctors
    WHERE doctor_id = 1  -- starting doctor

    UNION ALL

    -- Recursive case: follow referrals
    SELECT
        dr.referred_id,
        rc.referred_doctor,
        rc.depth + 1,
        rc.path || dr.referred_id
    FROM doctor_referrals dr
    JOIN referral_chain rc ON rc.referred_doctor = dr.referrer_id
    WHERE rc.depth < 5  -- prevent infinite loops
      AND NOT (dr.referred_id = ANY(rc.path))  -- prevent cycles
)
SELECT
    rc.referred_doctor,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    rc.depth AS referral_depth
FROM referral_chain rc
JOIN doctors d ON d.doctor_id = rc.referred_doctor
ORDER BY rc.depth;
```

Q25 — CTE for Readability: Top Hospital by Completion Rate

sql

```sql
WITH hospital_stats AS (
    SELECT
        h.hospital_id,
        h.hospital_name,
        COUNT(a.appointment_id) AS total_appointments,
        COUNT(*) FILTER (WHERE a.status = 'completed')  AS completed,
        COUNT(*) FILTER (WHERE a.status = 'cancelled')  AS cancelled,
        COUNT(*) FILTER (WHERE a.status = 'no_show')    AS no_shows
    FROM hospitals h
    JOIN doctors d ON d.hospital_id = h.hospital_id
    JOIN appointments a ON a.doctor_id = d.doctor_id
    GROUP BY h.hospital_id, h.hospital_name
),
hospital_rates AS (
    SELECT
        *,
        ROUND(100.0 * completed / NULLIF(total_appointments, 0), 2) AS completio
        ROUND(100.0 * cancelled / NULLIF(total_appointments, 0), 2) AS cancellat
    FROM hospital_stats
    WHERE total_appointments >= 50  -- only hospitals with meaningful data
)
SELECT *
FROM hospital_rates
ORDER BY completion_rate DESC;
```

7.4 Grouping, Aggregation & HAVING (Q26–Q35)

Q26 — GROUP BY with Multiple Aggregates: Doctor Performance Summary

```sql
```

```sql
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name)        AS doctor_name,
    COUNT(a.appointment_id)                         AS total_appointments,
    COUNT(*) FILTER (WHERE a.status = 'completed')  AS completed,
    COUNT(*) FILTER (WHERE a.status = 'cancelled')  AS cancelled,
    COUNT(*) FILTER (WHERE a.status = 'no_show')    AS no_shows,
    COUNT(DISTINCT a.patient_id)                    AS unique_patients,
    ROUND(AVG(d.consultation_fee), 2)               AS fee,
    SUM(d.consultation_fee)
        FILTER (WHERE a.status = 'completed')        AS total_revenue
FROM doctors d
LEFT JOIN appointments a ON a.doctor_id = d.doctor_id
GROUP BY d.doctor_id, d.first_name, d.last_name, d.consultation_fee
ORDER BY total_revenue DESC NULLS LAST;
```

Q27 — HAVING with Subquery: Hospitals Where Average Fee Exceeds Overall Average

sql

```sql
SELECT
    h.hospital_name,
    h.city,
    ROUND(AVG(d.consultation_fee), 2) AS avg_fee_at_hospital,
    COUNT(DISTINCT d.doctor_id)        AS doctor_count
FROM hospitals h
JOIN doctors d ON d.hospital_id = h.hospital_id
GROUP BY h.hospital_id, h.hospital_name, h.city
HAVING AVG(d.consultation_fee) > (SELECT AVG(consultation_fee) FROM doctors)
ORDER BY avg_fee_at_hospital DESC;
```

Q28 — Conditional Aggregation: Appointment Counts by Status per Doctor per Month

sql

```sql
SELECT
    EXTRACT(YEAR FROM a.appointment_date)  AS year,
    EXTRACT(MONTH FROM a.appointment_date) AS month,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    COUNT(*) FILTER (WHERE a.status = 'scheduled')  AS scheduled,
    COUNT(*) FILTER (WHERE a.status = 'completed')  AS completed,
    COUNT(*) FILTER (WHERE a.status = 'cancelled')  AS cancelled,
    COUNT(*) FILTER (WHERE a.status = 'no_show')    AS no_show,
    COUNT(*)                                         AS total
FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
GROUP BY year, month, d.doctor_id, d.first_name, d.last_name
ORDER BY year, month, doctor_name;
```

Q29 — ROLLUP: Hierarchical Aggregation (Hospital → Doctor → Total)

sql

```sql
SELECT
    COALESCE(h.hospital_name, 'ALL HOSPITALS')       AS hospital,
    COALESCE(CONCAT(d.first_name, ' ', d.last_name), 'ALL DOCTORS') AS doctor,
    COUNT(a.appointment_id)   AS appointments,
    SUM(d.consultation_fee)
        FILTER (WHERE a.status = 'completed') AS revenue
FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN hospitals h ON h.hospital_id = d.hospital_id
GROUP BY ROLLUP(h.hospital_name, (d.first_name, d.last_name))
ORDER BY hospital, doctor;
```

Q30 — CUBE: All Combinations of Hospital, Specialization, Month

sql

```sql
SELECT
    COALESCE(h.hospital_name, 'ALL')     AS hospital,
    COALESCE(s.name, 'ALL')              AS specialization,
    COALESCE(CAST(EXTRACT(MONTH FROM a.appointment_date) AS TEXT), 'ALL') AS mont
    COUNT(a.appointment_id)              AS appointment_count,
    SUM(d.consultation_fee)
        FILTER (WHERE a.status = 'completed') AS total_revenue
FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN hospitals h ON h.hospital_id = d.hospital_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
WHERE EXTRACT(YEAR FROM a.appointment_date) = 2024
GROUP BY CUBE(h.hospital_name, s.name, EXTRACT(MONTH FROM a.appointment_date))
ORDER BY hospital, specialization, month;
```

Q31 — GROUPING SETS: Custom Aggregation Combinations

sql

```sql
-- Get totals by city alone, by specialization alone, and by city+specialization
-- all in one query using GROUPING SETS:
SELECT
    h.city,
    s.name AS specialization,
    COUNT(a.appointment_id) AS total_appointments,
    SUM(d.consultation_fee) FILTER (WHERE a.status='completed') AS revenue
FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN hospitals h ON h.hospital_id = d.hospital_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
GROUP BY GROUPING SETS (
    (h.city),
    (s.name),
    (h.city, s.name)
)
ORDER BY h.city, s.name;
```

Q32 — STRING_AGG: Aggregate Prescriptions Per Appointment

sql

```sql
SELECT
    a.appointment_id,
    CONCAT(p.first_name, ' ', p.last_name)  AS patient_name,
    a.appointment_date,
    STRING_AGG(pr.medication_name, ', ' ORDER BY pr.prescription_id) AS medicatio
    COUNT(pr.prescription_id) AS prescription_count
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
LEFT JOIN prescriptions pr ON pr.appointment_id = a.appointment_id
WHERE a.status = 'completed'
GROUP BY a.appointment_id, p.first_name, p.last_name, a.appointment_date
HAVING COUNT(pr.prescription_id) > 0
ORDER BY prescription_count DESC;
```

Q33 — PERCENTILE_CONT: Finding Median Consultation Fee by Specialization

sql

```sql
SELECT
    s.name AS specialization,
    COUNT(DISTINCT d.doctor_id)                                    AS doctor_count
    MIN(d.consultation_fee)                                        AS min_fee,
    ROUND(AVG(d.consultation_fee), 2)                              AS avg_fee,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY d.consultation_fee) AS median_fee
    MAX(d.consultation_fee)                                        AS max_fee
FROM specializations s
JOIN doctor_specializations ds ON ds.specialization_id = s.specialization_id
JOIN doctors d ON d.doctor_id = ds.doctor_id
GROUP BY s.specialization_id, s.name
ORDER BY median_fee DESC;
```

Q34 — DATE Aggregations: Weekly and Monthly Appointment Trends

sql

```sql
SELECT
    DATE_TRUNC('week', appointment_date)::DATE AS week_start,
    COUNT(*)                                   AS total_bookings,
    COUNT(*) FILTER (WHERE status = 'completed') AS completions,
    COUNT(*) FILTER (WHERE status = 'cancelled') AS cancellations,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE status = 'cancelled')
        / NULLIF(COUNT(*), 0)
    , 2) AS cancellation_rate_pct
FROM appointments
WHERE appointment_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY week_start
ORDER BY week_start;
```

Q35 — HAVING with COUNT DISTINCT: Patients Visiting Multiple Specializations

sql

```sql
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name)  AS patient_name,
    COUNT(DISTINCT s.specialization_id)      AS distinct_specializations_visited,
    STRING_AGG(DISTINCT s.name, ', ' ORDER BY s.name) AS specializations
FROM patients p
JOIN appointments a ON a.patient_id = p.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
WHERE a.status = 'completed'
GROUP BY p.patient_id, p.first_name, p.last_name
HAVING COUNT(DISTINCT s.specialization_id) >= 3
ORDER BY distinct_specializations_visited DESC;
```

7.5 Window Functions (Q36–Q45)

> Window functions allow you to perform calculations across a "window" (set of related rows) without collapsing them into a group. The key idea: the original rows remain visible in the result, but each row gets a computed value based on its surrounding window.

Q36 — ROW_NUMBER: Assign Sequential Numbers Within Each Doctor's Appointments

```sql
SELECT
    appointment_id,
    patient_id,
    doctor_id,
    appointment_date,
    status,
    ROW_NUMBER() OVER (
        PARTITION BY doctor_id
        ORDER BY appointment_date, appointment_time
    ) AS appointment_sequence_for_doctor
FROM appointments
ORDER BY doctor_id, appointment_date;
```

Q37 — RANK vs. DENSE_RANK: Doctor Rankings by Revenue (Illustrating the Difference)

```sql
SELECT
    doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    total_revenue,
    RANK()       OVER (ORDER BY total_revenue DESC) AS rank_with_gaps,
    DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS rank_no_gaps,
    ROW_NUMBER() OVER (ORDER BY total_revenue DESC) AS row_num
FROM (
    SELECT
        d.doctor_id,
        d.first_name,
        d.last_name,
        SUM(d.consultation_fee) FILTER (WHERE a.status = 'completed') AS total_re
    FROM doctors d
    LEFT JOIN appointments a ON a.doctor_id = d.doctor_id
    GROUP BY d.doctor_id, d.first_name, d.last_name
) d
ORDER BY total_revenue DESC;
-- If two doctors tie at rank 1: RANK gives 1,1,3; DENSE_RANK gives 1,1,2; ROW_NU
```

Q38 — LAG and LEAD: Compare Each Appointment Date to Previous/Next for a Patient

```sql
```

```sql
SELECT
    patient_id,
    appointment_id,
    appointment_date,
    status,
    LAG(appointment_date)  OVER (PARTITION BY patient_id ORDER BY appointment_da
    LEAD(appointment_date) OVER (PARTITION BY patient_id ORDER BY appointment_da
    appointment_date -
        LAG(appointment_date) OVER (PARTITION BY patient_id ORDER BY appointment_
        AS days_since_last_visit
FROM appointments
ORDER BY patient_id, appointment_date;
```

Q39 — SUM() OVER with Running Total: Cumulative Revenue Per Doctor

sql

```sql
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    a.appointment_date,
    d.consultation_fee AS appointment_revenue,
    SUM(d.consultation_fee) OVER (
        PARTITION BY d.doctor_id
        ORDER BY a.appointment_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_revenue_for_doctor,
    SUM(d.consultation_fee) OVER (
        ORDER BY a.appointment_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total_all_doctors
FROM appointments a
JOIN doctors d ON d.doctor_id = a.doctor_id
WHERE a.status = 'completed'
ORDER BY a.appointment_date;
```

Q40 — AVG() OVER with Moving Window: 7-Day Rolling Average Appointments

sql

```sql
SELECT
    appointment_date,
    daily_count,
    ROUND(AVG(daily_count) OVER (
        ORDER BY appointment_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ), 2) AS rolling_7day_avg
FROM (
    SELECT appointment_date, COUNT(*) AS daily_count
    FROM appointments
    GROUP BY appointment_date
) daily
ORDER BY appointment_date;
```

Q41 — NTILE: Divide Doctors into Quartiles by Fee

sql

```sql
SELECT
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    d.consultation_fee,
    NTILE(4) OVER (ORDER BY d.consultation_fee) AS fee_quartile,
    CASE NTILE(4) OVER (ORDER BY d.consultation_fee)
        WHEN 1 THEN 'Bottom 25%'
        WHEN 2 THEN 'Lower-Mid 25%'
        WHEN 3 THEN 'Upper-Mid 25%'
        WHEN 4 THEN 'Top 25%'
    END AS fee_tier
FROM doctors d
ORDER BY d.consultation_fee;
```

Q42 — FIRST_VALUE and LAST_VALUE: Compare Each Doctor's Fee to Cheapest in Specialization

sql

```sql
SELECT
    s.name AS specialization,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    d.consultation_fee,
    FIRST_VALUE(d.consultation_fee) OVER (
        PARTITION BY s.specialization_id
        ORDER BY d.consultation_fee
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS cheapest_in_specialization,
    LAST_VALUE(d.consultation_fee) OVER (
        PARTITION BY s.specialization_id
        ORDER BY d.consultation_fee
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS most_expensive_in_specialization,
    d.consultation_fee -
        FIRST_VALUE(d.consultation_fee) OVER (
            PARTITION BY s.specialization_id
            ORDER BY d.consultation_fee
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        ) AS premium_over_cheapest
FROM doctors d
JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
JOIN specializations s ON s.specialization_id = ds.specialization_id
ORDER BY s.name, d.consultation_fee;
```

Q43 — PERCENT_RANK and CUME_DIST: Percentile Position of Each Doctor's Fee

sql

```sql
SELECT
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    d.consultation_fee,
    ROUND(PERCENT_RANK() OVER (ORDER BY d.consultation_fee)::NUMERIC * 100, 2) AS
    ROUND(CUME_DIST()    OVER (ORDER BY d.consultation_fee)::NUMERIC * 100, 2) AS
FROM doctors d
ORDER BY d.consultation_fee;
-- PERCENT_RANK: (rank - 1) / (total rows - 1); ranges 0 to 1
-- CUME_DIST: how many rows are <= current row / total rows; ranges 1/n to 1
```

Q44 — Top N Per Group Using ROW_NUMBER: Top 2 Doctors Per Specialization by Revenue

sql

```sql
WITH doctor_revenue AS (
    SELECT
        d.doctor_id,
        CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
        s.name AS specialization,
        SUM(d.consultation_fee) FILTER (WHERE a.status = 'completed') AS total_re
        ROW_NUMBER() OVER (
            PARTITION BY s.specialization_id
            ORDER BY SUM(d.consultation_fee) FILTER (WHERE a.status = 'completed'
        ) AS rank_within_specialization
    FROM doctors d
    JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
    JOIN specializations s ON s.specialization_id = ds.specialization_id
    LEFT JOIN appointments a ON a.doctor_id = d.doctor_id
    GROUP BY d.doctor_id, d.first_name, d.last_name, s.specialization_id, s.name
)
SELECT specialization, doctor_name, total_revenue, rank_within_specialization
FROM doctor_revenue
WHERE rank_within_specialization <= 2
ORDER BY specialization, rank_within_specialization;
```

Q45 — Session Gap Analysis: Detect Gaps Between Patient Visits

sql

```sql
WITH patient_visits AS (
    SELECT
        patient_id,
        appointment_date,
        LAG(appointment_date) OVER (
            PARTITION BY patient_id
            ORDER BY appointment_date
        ) AS prev_visit
    FROM appointments
    WHERE status = 'completed'
),
gap_analysis AS (
    SELECT
        patient_id,
        appointment_date,
        prev_visit,
        appointment_date - prev_visit AS days_gap
    FROM patient_visits
    WHERE prev_visit IS NOT NULL
)
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    COUNT(*) AS visit_count,
    MAX(ga.days_gap) AS longest_gap_days,
    ROUND(AVG(ga.days_gap), 1) AS avg_gap_days,
    CASE
        WHEN MAX(ga.days_gap) > 365 THEN 'High Risk - Long Gaps'
        WHEN MAX(ga.days_gap) > 180 THEN 'Moderate Risk'
        ELSE 'Regular Visitor'
    END AS patient_visit_pattern
FROM gap_analysis ga
JOIN patients p ON p.patient_id = ga.patient_id
GROUP BY p.patient_id, p.first_name, p.last_name
ORDER BY longest_gap_days DESC;
```

7.6 Triggers, Views & EXPLAIN Plans (Q46–Q50)

Q46 — CREATE VIEW: Appointment Detail View

sql

```sql
-- Create a reusable view for full appointment details:
CREATE OR REPLACE VIEW vw_appointment_details AS
SELECT
    a.appointment_id,
    a.appointment_date,
    a.appointment_time,
    a.status,
    a.reason_for_visit,
    a.consultation_notes,
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    p.phone AS patient_phone,
    p.email AS patient_email,
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    d.consultation_fee,
    h.hospital_name,
    h.city AS hospital_city,
    STRING_AGG(s.name, ', ') AS specializations
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN hospitals h ON h.hospital_id = d.hospital_id
LEFT JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
LEFT JOIN specializations s ON s.specialization_id = ds.specialization_id
GROUP BY a.appointment_id, a.appointment_date, a.appointment_time, a.status,
         a.reason_for_visit, a.consultation_notes,
         p.patient_id, p.first_name, p.last_name, p.phone, p.email,
         d.doctor_id, d.first_name, d.last_name, d.consultation_fee,
         h.hospital_name, h.city;

-- Usage:
SELECT * FROM vw_appointment_details WHERE hospital_city = 'Mumbai' AND status =
```

Q47 — TRIGGER: Prevent Double-Booking and Log Changes

sql

```sql
-- Function to prevent overlapping appointments for the same doctor:
CREATE OR REPLACE FUNCTION prevent_double_booking()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM appointments
        WHERE doctor_id = NEW.doctor_id
          AND appointment_date = NEW.appointment_date
          AND appointment_time = NEW.appointment_time
          AND appointment_id <> COALESCE(NEW.appointment_id, -1)
          AND status NOT IN ('cancelled')
    ) THEN
        RAISE EXCEPTION 'Doctor % already has an appointment at % on %',
            NEW.doctor_id, NEW.appointment_time, NEW.appointment_date;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_double_booking
BEFORE INSERT OR UPDATE ON appointments
FOR EACH ROW
EXECUTE FUNCTION prevent_double_booking();

-- Audit log trigger:
CREATE TABLE appointment_audit_log (
    log_id       SERIAL PRIMARY KEY,
    appointment_id INT,
    action       VARCHAR(10),
    old_status   VARCHAR(20),
    new_status   VARCHAR(20),
    changed_at   TIMESTAMPTZ DEFAULT NOW(),
    changed_by   TEXT DEFAULT CURRENT_USER
);

CREATE OR REPLACE FUNCTION log_appointment_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND OLD.status <> NEW.status THEN
        INSERT INTO appointment_audit_log (appointment_id, action, old_status, ne
        VALUES (NEW.appointment_id, 'STATUS_CHANGE', OLD.status, NEW.status);
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO appointment_audit_log (appointment_id, action, old_status, ne
        VALUES (OLD.appointment_id, 'DELETE', OLD.status, NULL);
```

```
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_appointments
AFTER UPDATE OR DELETE ON appointments
FOR EACH ROW
EXECUTE FUNCTION log_appointment_changes();
```

Q48 — MATERIALIZED VIEW with Index: Performance-Optimized Analytics

sql

```sql
-- Materialized view for a pre-computed doctor performance dashboard:
CREATE MATERIALIZED VIEW mv_doctor_performance AS
SELECT
    d.doctor_id,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
    h.hospital_name,
    h.city,
    STRING_AGG(DISTINCT s.name, ', ')                          AS specializations
    COUNT(a.appointment_id)                                    AS total_appointme
    COUNT(*) FILTER (WHERE a.status = 'completed')             AS completed,
    COUNT(*) FILTER (WHERE a.status = 'cancelled')             AS cancelled,
    COUNT(DISTINCT a.patient_id)                               AS unique_patients
    SUM(d.consultation_fee) FILTER (WHERE a.status = 'completed') AS total_revenu
    ROUND(AVG(d.consultation_fee), 2)                          AS avg_fee,
    ROUND(100.0 * COUNT(*) FILTER (WHERE a.status = 'completed')
          / NULLIF(COUNT(*), 0), 2)                            AS completion_rate
FROM doctors d
JOIN hospitals h ON h.hospital_id = d.hospital_id
LEFT JOIN appointments a ON a.doctor_id = d.doctor_id
LEFT JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
LEFT JOIN specializations s ON s.specialization_id = ds.specialization_id
GROUP BY d.doctor_id, d.first_name, d.last_name, h.hospital_name, h.city, d.consu
WITH DATA;

-- Index the materialized view for fast lookups:
CREATE INDEX idx_mv_doctor_perf_city ON mv_doctor_performance(city);
CREATE INDEX idx_mv_doctor_perf_revenue ON mv_doctor_performance(total_revenue DE

-- Refresh the materialized view (can be scheduled via pg_cron):
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_doctor_performance;
-- CONCURRENTLY allows reads during refresh but requires a unique index:
CREATE UNIQUE INDEX idx_mv_doctor_perf_id ON mv_doctor_performance(doctor_id);
```

Q49 — EXPLAIN ANALYZE: Diagnosing and Optimizing a Slow Query

sql

```sql
-- First, run EXPLAIN ANALYZE to see the query plan:
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    COUNT(a.appointment_id) AS total_appointments
FROM patients p
LEFT JOIN appointments a ON a.patient_id = p.patient_id
WHERE p.city = 'Mumbai'
  AND a.appointment_date >= '2024-01-01'
GROUP BY p.patient_id, p.first_name, p.last_name
HAVING COUNT(a.appointment_id) > 5;

/*
Sample EXPLAIN output to understand:
Seq Scan on patients (cost=0.00..892.50 rows=1250 ...)
  Filter: (city = 'Mumbai')                          ← Full scan! Need index
Hash Join (cost=...)
  Hash Cond: (a.patient_id = p.patient_id)
  -> Seq Scan on appointments (cost=...)             ← Full scan on appointments
       Filter: (appointment_date >= '2024-01-01')    ← Need index on date

FIX: Add indexes:
*/
CREATE INDEX idx_patients_city ON patients(city);
CREATE INDEX idx_appts_date_patient ON appointments(appointment_date, patient_id);

-- After indexes, re-run EXPLAIN to confirm Index Scans replace Seq Scans:
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT p.patient_id, ... -- same query
-- Should now show: Index Scan using idx_patients_city
```

Q50 — Advanced: Pivot Table Using CROSSTAB (Monthly Revenue by Specialization)

sql

```sql
-- Install the tablefunc extension (needed for crosstab):
CREATE EXTENSION IF NOT EXISTS tablefunc;

-- Query to pivot monthly revenue by specialization:
SELECT *
FROM crosstab(
    -- Row query (must be ordered by row_name, column_name):
    $$
    SELECT
        s.name AS specialization,
        TO_CHAR(a.appointment_date, 'YYYY-MM') AS month,
        SUM(d.consultation_fee)::NUMERIC AS revenue
    FROM appointments a
    JOIN doctors d ON d.doctor_id = a.doctor_id
    JOIN doctor_specializations ds ON ds.doctor_id = d.doctor_id
    JOIN specializations s ON s.specialization_id = ds.specialization_id
    WHERE a.status = 'completed'
      AND EXTRACT(YEAR FROM a.appointment_date) = 2024
    GROUP BY s.name, TO_CHAR(a.appointment_date, 'YYYY-MM')
    ORDER BY 1, 2
    $$,
    -- Column category query:
    $$
    SELECT DISTINCT TO_CHAR(appointment_date, 'YYYY-MM')
    FROM appointments
    WHERE EXTRACT(YEAR FROM appointment_date) = 2024
    ORDER BY 1
    $$
) AS ct (
    specialization TEXT,
    "2024-01" NUMERIC, "2024-02" NUMERIC, "2024-03" NUMERIC,
    "2024-04" NUMERIC, "2024-05" NUMERIC, "2024-06" NUMERIC,
    "2024-07" NUMERIC, "2024-08" NUMERIC, "2024-09" NUMERIC,
    "2024-10" NUMERIC, "2024-11" NUMERIC, "2024-12" NUMERIC
);
```

8. Top 50 Placement Interview Questions

> Each answer is a detailed 3-5 sentence expert response. Memorize the concepts; internalize the reasoning.

Q1. What is the difference between a Primary Key and a Unique Key?

A **Primary Key** uniquely identifies each row in a table and enforces both uniqueness and NOT NULL — a primary key column can never contain a NULL value. A **Unique Key** also enforces uniqueness but allows exactly one NULL value (since NULL ≠ NULL in SQL's three-valued logic). A table can have only ONE primary key but can have multiple unique keys. Additionally, the primary key is the default target for foreign key references from other tables, while unique keys can also be referenced but are used less commonly in this way.

Q2. What is the difference between DELETE, TRUNCATE, and DROP?

`DELETE` is a DML (Data Manipulation Language) command that removes rows one by one based on an optional WHERE clause, fires row-level DELETE triggers, and can be rolled back within a transaction. `TRUNCATE` is a DDL command that removes ALL rows from a table extremely fast (by deallocating data pages rather than deleting row-by-row), does NOT fire row-level triggers, and in PostgreSQL CAN be rolled back within a transaction (unlike in MySQL). `DROP` removes the entire table structure — the table, its data, its indexes, and its constraints — permanently, and cannot be rolled back without a backup. Rule of thumb: DELETE when you need selective removal; TRUNCATE when you need to empty a large table fast; DROP when you want to eliminate the table entirely.

Q3. Explain INNER JOIN vs. LEFT JOIN vs. FULL OUTER JOIN.

An **INNER JOIN** returns only the rows where the join condition is satisfied in BOTH tables — rows from either table with no match are excluded. A **LEFT JOIN** (LEFT OUTER JOIN) returns ALL rows from the LEFT table plus matching rows from the right table; where there's no match, right-side columns are NULL — useful for "show me everything, including items with no related records." A **FULL OUTER JOIN** returns ALL rows from BOTH tables, filling unmatched columns with NULL — useful for set reconciliation like "show me all doctors and all patients, even if some doctors have no patients and some patients have no doctors." Choosing the correct join type is one of the most common sources of subtle query bugs.

Q4. What is a Correlated Subquery? How does it differ from a non-correlated subquery?

A **non-correlated (nested) subquery** runs independently of the outer query — it is executed once, produces a result set, and the outer query uses that result. Example: `WHERE consultation_fee > (SELECT AVG(consultation_fee) FROM doctors)` — the inner query runs once. A **correlated subquery** references a column from the outer query, so it must be executed once for EACH ROW of the outer query. Example: finding each doctor's most recent appointment requires a subquery that references `outer_query.doctor_id` — it re-runs for every doctor row. Correlated subqueries can be slow on large tables because of this N×inner_query complexity; they are often replaceable with window functions (like `ROW_NUMBER()` or `FIRST_VALUE()`), which are much more efficient.

Q5. What are window functions? How are RANK, DENSE_RANK, and ROW_NUMBER different?

Window functions perform calculations across a set of rows (a "window") that are related to the current row, WITHOUT collapsing them into a single result like GROUP BY does. The `OVER()` clause defines the window — `PARTITION BY` divides rows into groups and `ORDER BY` sets the order within each partition. **ROW_NUMBER** assigns a unique sequential number to each row — no ties, ever. **RANK** assigns the same number to tied rows but then skips numbers (1,1,3 if two rows tie for first). **DENSE_RANK** also assigns the same number to ties but does NOT skip (1,1,2). Use RANK when you want to know "exact position including gaps" (like sports rankings); use DENSE_RANK when you want to know "how many distinct ranks are below me."

Q6. What is normalization? What problem does it solve?

Normalization is the systematic process of structuring a relational database to reduce data redundancy and eliminate update, insertion, and deletion anomalies. Without normalization, the same data is stored in multiple places — changing it requires updating every copy, and missing even one creates inconsistency. Normalization decomposes one large "fat" table into multiple smaller, logically coherent tables connected by foreign keys, ensuring each fact is stored exactly once. The tradeoff is that reads become slightly more expensive (requiring joins) while writes become safer and more maintainable. In practice, most production systems normalize to 3NF or BCNF for OLTP workloads.

Q7. What is the difference between 2NF and 3NF?

Both forms deal with different types of dependencies that violate the principle of "one fact in one place." **2NF** addresses **partial dependencies** — non-key attributes that depend on only PART of a composite primary key (not the full key). If the primary key is a single column, a table in 1NF is automatically in 2NF. **3NF** addresses **transitive dependencies** — non-key attributes that depend on OTHER non-key attributes (i.e., non-key → non-key). A table could be in 2NF but violate 3NF if, for example, `zip_code → city` where both are non-key columns. The formal 3NF rule is: for every non-trivial FD X→A, either X is a superkey OR A is a prime attribute.

Q8. What is BCNF and when does 3NF not suffice?

BCNF (Boyce-Codd Normal Form) is a stricter version of 3NF. While 3NF allows a non-superkey to determine a prime attribute (part of a candidate key), BCNF forbids this entirely — EVERY non-trivial functional dependency X→Y must have X as a superkey. The gap between 3NF and BCNF only appears when a relation has multiple overlapping candidate keys. The classic example is a scheduling relation where `{Doctor, Specialization}` and `{Doctor, Coordinator}` are both candidate keys, but `Coordinator → Specialization` exists and Coordinator is not a superkey. BCNF decomposition might lose some FDs, which is why 3NF is sometimes preferred as a decomposition target — it guarantees both lossless join AND dependency preservation.

Q9. Explain ACID properties with a real-world example.

ACID stands for Atomicity, Consistency, Isolation, and Durability — the four guarantees that make database transactions reliable. In a hospital appointment system: **Atomicity** ensures that booking an appointment (creating the record + deducting a fee + blocking the slot) either ALL happens or ALL gets rolled back — you never end up with a charge but no appointment. **Consistency** ensures that the unique constraint on (doctor, date, time) is always enforced — no double-booking can ever exist in the database. **Isolation** ensures that two patients booking simultaneously appear to happen sequentially — one succeeds and the other sees the slot as taken. **Durability** ensures that once the patient receives a "booking confirmed" message, the appointment record will survive even if the server crashes one second later.

Q10. What are the four Transaction Isolation Levels?

The SQL standard defines four isolation levels, each preventing a different set of anomalies. **Read Uncommitted** (most permissive) allows dirty reads — a transaction

can see uncommitted changes from other transactions. **Read Committed** (PostgreSQL default) prevents dirty reads but allows non-repeatable reads — the same SELECT can return different values if another transaction commits between the two reads. **Repeatable Read** prevents dirty reads and non-repeatable reads — a transaction's reads are consistent throughout its lifetime based on a snapshot taken at its start. **Serializable** (most strict) prevents all anomalies including phantom reads — transactions appear to run completely serially. Higher isolation = fewer anomalies = lower concurrency throughput.

Q11. What is a deadlock? How do databases handle it?

A deadlock occurs when two or more transactions are each waiting for a lock held by another transaction in the set, forming a circular dependency that can never be resolved without external intervention. For example, T1 holds a lock on the `appointments` table and waits for `doctors`, while T2 holds the lock on `doctors` and waits for `appointments`. Databases detect deadlocks by periodically analyzing a **wait-for graph** — a directed graph where edges represent "is waiting for" relationships; a cycle indicates a deadlock. The DBMS selects a "victim" transaction (typically the one that has done the least work or has the lowest priority), aborts it (releases its locks), and allows the others to proceed. Applications must handle the resulting rollback error by retrying the transaction.

Q12. What is MVCC (Multi-Version Concurrency Control)?

MVCC is a concurrency control technique where the database maintains multiple versions of each data row simultaneously, allowing readers and writers to operate on different versions without blocking each other. In PostgreSQL, every row has hidden `xmin` (transaction that created it) and `xmax` (transaction that deleted/updated it) timestamps. When a transaction reads a row, it sees the version that was committed at or before the transaction's start time — regardless of what concurrent writers are doing. This eliminates the classic read-write lock conflict: readers never block writers and writers never block readers. The trade-off is storage bloat from dead row versions, which PostgreSQL's VACUUM process periodically cleans up.

Q13. How does indexing improve query performance? What are the downsides?

An index is a separate data structure (usually a B+ Tree in PostgreSQL) that maps column values to the physical location of rows on disk. Without an index, a query `WHERE email = 'riya@example.com'` requires a full table scan — reading every row. With an index on `email`, the database traverses the B+ Tree in O(log n) time and jumps directly

to the row. The performance benefit is dramatic for large tables and selective queries. **Downsides:** Every index adds overhead to INSERT, UPDATE, and DELETE operations (the index must be maintained alongside the table). Indexes consume additional disk space. Too many indexes can paradoxically slow down write-heavy systems. The query optimizer may also sometimes choose an incorrect index — always use `EXPLAIN ANALYZE` to verify.

Q14. What is the difference between a B-Tree and a B+ Tree?

In a **B-Tree**, data pointers (actual record references) can be stored in any node — root, internal, or leaf. In a **B+ Tree**, all data pointers are stored ONLY in leaf nodes; internal nodes contain only keys that serve as navigation guides. This distinction has major performance implications: B+ Trees have higher fanout (more keys per internal node since they don't store data), resulting in shallower trees and fewer disk I/Os. More critically, B+ Tree leaf nodes are connected in a doubly-linked list, enabling blazing-fast range queries — you find the start of a range and follow the linked list without re-traversing the tree. These two advantages — shorter trees and sequential range scans — are why virtually every production database uses B+ Trees.

Q15. How would you optimize a slow-running query?

Start with `EXPLAIN ANALYZE` to see the actual execution plan — identify if there are Sequential Scans on large tables (indicates missing indexes) or Nested Loop joins on large datasets (may need Hash Join). First, add appropriate indexes: if the WHERE clause filters on a column, index it; if the query joins on a column, index it on both tables. Check if the query can use a **covering index** (index includes all selected columns, enabling index-only scans). Rewrite correlated subqueries as JOINs or window functions. Use CTEs for readability but be aware that in older PostgreSQL (pre-12), CTEs act as optimization fences — materialized even when unnecessary. Consider partitioning large tables and using `VACUUM ANALYZE` to keep statistics fresh. Finally, for repeated heavy queries, consider materialized views.

Q16. What is the difference between WHERE and HAVING?

`WHERE` filters rows **before** grouping and aggregation — it operates on individual rows and cannot reference aggregate functions. `HAVING` filters **after** grouping — it operates on aggregated results and can use aggregate functions. Example: `WHERE status = 'completed'` filters individual appointment rows before any grouping. `HAVING COUNT(*) > 10` filters groups (e.g., doctors) based on their aggregate count after GROUP BY. A common mistake is trying to write `WHERE COUNT(*) > 10` — this fails because COUNT

doesn't exist at the WHERE stage. If a filter doesn't need aggregation, always put it in WHERE (filtering early reduces the number of rows that need to be aggregated, which is faster).

Q17. What is a subquery? Explain its types.

A **subquery** is a query nested inside another query. There are several types: A **scalar subquery** returns exactly one row and one column and can appear anywhere an expression is expected. A **row subquery** returns one row with multiple columns. A **table subquery** returns a table (multiple rows, multiple columns) and is used in FROM clauses as a "derived table." A **correlated subquery** references columns from the outer query and re-executes for each row of the outer query. A **non-correlated subquery** runs once independently. Subqueries with `IN`, `NOT IN`, `EXISTS`, `NOT EXISTS`, `ANY`, and `ALL` are common patterns. For performance, `EXISTS` is usually faster than `IN` when checking for existence because it short-circuits on the first match.

Q18. What is a VIEW? What are its limitations?

A VIEW is a **virtual table** defined by a stored SQL query. When you query a view, PostgreSQL runs the underlying query and returns the result — no data is physically stored (unlike a materialized view). Views serve multiple purposes: security (exposing only certain columns to certain users), simplicity (hiding complex joins behind a simple name), and logical abstraction (allowing schema changes without breaking applications). **Limitations:** Views cannot be directly updated in most cases (they're read-only unless they're "simple" views satisfying strict rules or you use INSTEAD OF triggers). Views do not store data so they run their underlying query every time they're accessed (performance overhead for complex views). Circular views (view A references view B which references view A) are not allowed.

Q19. What is a MATERIALIZED VIEW? When would you use it?

A **materialized view** is like a regular view but with a physical snapshot of the result stored on disk. Unlike a regular view that re-executes its query on every access, a materialized view caches the result until explicitly refreshed with `REFRESH MATERIALIZED VIEW`. Use it when: the underlying query is expensive (multiple joins, heavy aggregations), the data doesn't need to be real-time (tolerates staleness), and the same result is queried frequently. In MediBook, a materialized view for doctor performance metrics (total revenue, completion rate, unique patients) might refresh nightly. The `CONCURRENTLY` option allows reads during refresh but requires a unique index. The key trade-off: faster reads at the cost of potential data staleness.

Q20. What is a Stored Procedure vs. a Function in PostgreSQL?

In PostgreSQL (version 11+), **stored procedures** are invoked with `CALL procedure_name()` and can manage transactions internally — they can include `COMMIT` and `ROLLBACK` within the procedure body. **Functions** are invoked with `SELECT function_name()` and cannot contain transaction control statements — they run within the caller's transaction. Functions must return a value (or VOID); procedures have no return value requirement. Both can contain PL/pgSQL logic, loops, conditionals, and SQL statements. Use a procedure when you need transaction management within the routine; use a function when you need a reusable computation that returns a result.

Q21. What is a trigger? Describe a real use case.

A trigger is a special stored function that automatically executes in response to specific database events (INSERT, UPDATE, DELETE, TRUNCATE) on a table. Triggers can fire BEFORE, AFTER, or INSTEAD OF the triggering event, and can operate on each row (FOR EACH ROW) or once per statement (FOR EACH STATEMENT). **Real use case:** In MediBook, a BEFORE INSERT trigger checks if the new appointment's `(doctor_id, date, time)` combination already exists — if so, it raises an exception to prevent double-booking, all without any application-level code. An AFTER UPDATE trigger writes to an `appointment_audit_log` table whenever an appointment's status changes. Triggers are powerful but should be used sparingly — they add hidden behavior that makes debugging and testing harder.

Q22. Explain the CAP Theorem in plain language.

The CAP Theorem states that any distributed data system can provide at most TWO of three guarantees: **Consistency** (every read gets the most recent write), **Availability** (every request gets a response, though possibly not the latest data), and **Partition Tolerance** (the system continues working despite network failures between nodes). Since network partitions are unavoidable in real distributed systems (hardware fails, cables break), the practical choice is between **CP** (sacrifice availability during a partition — system goes down rather than return stale data) and **AP** (sacrifice consistency during a partition — system stays up but might return outdated data). PostgreSQL in synchronous replication mode is CP; Cassandra/DynamoDB are AP. The choice depends on whether stale data is acceptable — never acceptable for financial transactions (CP), often acceptable for social feeds (AP).

Q23. What is Database Sharding? What are its challenges?

Sharding is the practice of horizontally distributing data across multiple database servers (shards), where each shard holds a distinct subset of the rows. A shard key determines which rows go to which shard (e.g., hash of `patient_id`, or geographic region). The primary benefit is linear horizontal scalability — adding more shards increases both storage and throughput capacity. **Challenges:** Cross-shard queries (especially JOINs) require querying multiple shards and merging results in the application layer — complex and slow. Re-sharding (adding new shards and rebalancing data) is operationally painful without careful design. Hotspots (one shard handling disproportionate traffic) can occur with poor shard key choices. Maintaining cross-shard transactions with ACID guarantees is very difficult. Sharding should be the last resort after exhausting vertical scaling and replication.

Q24. What is the difference between Clustered and Non-Clustered Indexes?

A **clustered index** determines the physical order of data rows on disk — the table's rows are sorted and stored according to the clustered index key. Because the data IS the index (or co-located with it), there's no need to follow a second pointer — the leaf node of the B+ tree directly contains the row. In PostgreSQL, `CLUSTER tablename USING indexname` physically reorders the table. There can be only ONE clustered index per table (data can only be sorted one way). A **non-clustered index** is a separate B+ Tree structure with keys pointing to the actual row's physical location (a heap tuple). Multiple non-clustered indexes can exist per table. Clustered indexes are fastest for range queries on the cluster key; non-clustered are faster for point lookups on non-clustered key columns.

Q25. How do you handle NULL values in SQL?

NULL represents the absence of a value (or an unknown value) — it is NOT zero, NOT empty string, NOT false. NULL propagates through arithmetic: `NULL + 5 = NULL`, `NULL * 0 = NULL`. Comparison with NULL always yields NULL (unknown): `NULL = NULL` is NULL (not TRUE), which is why `WHERE column = NULL` never returns rows — you must use `WHERE column IS NULL` or `WHERE column IS NOT NULL`. Aggregate functions like SUM, AVG, COUNT(column) ignore NULLs (COUNT(*) counts all rows including nulls; COUNT(column) ignores nulls). The functions `COALESCE(a, b, c)` returns the first non-NULL value, and `NULLIF(a, b)` returns NULL if a=b (useful to prevent division-by-zero: `total / NULLIF(denominator, 0)`). Understanding NULL behavior is essential for writing correct queries.

Q26. What is the difference between SQL and NoSQL databases?

SQL (relational) databases use a fixed schema, organize data in tables, use SQL as a query language, and provide strong ACID guarantees. They excel at complex queries involving multiple related entities, structured data with defined relationships, and workloads requiring consistency. NoSQL databases trade schema flexibility, scale, and speed at the cost of consistency guarantees and query expressiveness. Key-value stores (Redis) give O(1) lookups; document stores (MongoDB) allow flexible, nested JSON data; columnar stores (Cassandra) excel at write-heavy, analytically queried time-series data; graph databases (Neo4j) excel at traversing relationships. The common misconception is that NoSQL means "no schema" — in practice, your application enforces the schema implicitly. Choose SQL for consistency and rich queries; choose NoSQL for specific performance, scale, or flexibility needs.

Q27. What is Connection Pooling and why is it important?

Creating a new database connection involves TCP handshaking, authentication, memory allocation, and session initialization — a process that can take 20-100ms and consumes significant server resources. In a web application with thousands of concurrent requests, creating a new connection for each request would overwhelm both the application server and the database. **Connection pooling** maintains a pool of pre-established connections that are reused across requests. A client "borrows" a connection from the pool, uses it for the duration of the query/transaction, and returns it. Tools like PgBouncer (for PostgreSQL) can multiplex thousands of application connections over just 20-50 actual database connections. Without connection pooling, a moderate-traffic web application with 500 concurrent users might create 500 connections — exhausting PostgreSQL's `max_connections` and causing connection failures.

Q28. What is denormalization? When is it appropriate?

Denormalization is the intentional introduction of redundancy into a database schema — it is the OPPOSITE of normalization and is done deliberately to improve read performance. In a fully normalized schema, generating a report on appointment revenue by hospital requires joining 4-5 tables every time. By pre-computing and storing `hospital_name` and `specialization` directly in the `appointments` table (or a summary table), the report query becomes a simple GROUP BY with no joins. Denormalization is appropriate for: read-heavy OLAP/reporting workloads, analytical data warehouses (star/snowflake schemas), cached or materialized result sets, and anywhere query performance has been profiled and proven to be the bottleneck. Always start normalized; denormalize only with measurement-backed justification,

and document the decision — otherwise you'll create the very anomalies normalization was designed to prevent.

Q29. What is the difference between UNION and UNION ALL?

`UNION` combines the results of two SELECT queries and removes duplicate rows from the final result set. `UNION ALL` combines results and keeps ALL rows, including duplicates. `UNION ALL` is always faster because it doesn't need to sort and deduplicate the results. Use `UNION` when you genuinely need distinct results and duplicates would be problematic; use `UNION ALL` when you know there are no duplicates or when duplicates are acceptable (and always prefer it in that case for performance). Both require the SELECT queries to be **union-compatible**: same number of columns, and corresponding columns must have compatible data types. Column names in the result come from the first SELECT query.

Q30. Explain the concept of Foreign Keys and Referential Integrity.

A **foreign key** is a column (or set of columns) in a child table whose values must match values in the referenced primary/unique key of a parent table — or be NULL. This constraint enforces **referential integrity**: you cannot have an appointment (child) referencing a non-existent doctor (parent). The database enforces this automatically, preventing orphaned records. On DELETE/UPDATE of a parent row, the foreign key can be configured with actions: `CASCADE` (propagate the delete/update to children), `SET NULL` (set FK column to NULL in children), `SET DEFAULT` (set to default value), or `RESTRICT`/`NO ACTION` (prevent parent modification if children exist — the default). Foreign keys have a performance cost (index on FK column is essential) but provide data integrity that is very difficult to guarantee purely at the application level.

Q31. What is the SQL execution order?

SQL statements are WRITTEN in one order but PROCESSED in a different logical order. Understanding this is essential for writing correct queries and debugging errors. The logical execution order is:

1. `FROM` + `JOIN` — identify the source tables and build the initial row set
2. `WHERE` — filter rows (can reference columns but NOT aliases or aggregates)
3. `GROUP BY` — group remaining rows
4. `HAVING` — filter groups (can reference aggregates)
5. `SELECT` — compute expressions, apply window functions, assign aliases
6. `DISTINCT` — remove duplicates

7. `ORDER BY` — sort (can reference SELECT aliases — because it runs AFTER SELECT)
8. `LIMIT`/`OFFSET` — truncate the result

This explains why `WHERE COUNT(*) > 5` is invalid (WHERE runs before GROUP BY), why you can't use a SELECT alias in WHERE (alias created in step 5), but you CAN use it in ORDER BY (step 7).

Q32. What is SQL Injection? How do you prevent it?

SQL Injection is a critical security vulnerability where an attacker inserts malicious SQL code into input fields that are incorporated into database queries without proper sanitization. Classic example: a login form where the username input `' OR '1'='1` causes the query `SELECT * FROM users WHERE username='' OR '1'='1'` to return all users — bypassing authentication. Prevention strategies: **Parameterized queries/prepared statements** (the gold standard — the SQL structure and the data are sent separately, so user input can NEVER be interpreted as SQL), **ORMs** (provide parameterization by default), **stored procedures** (with parameterized inputs), **input validation**, and **principle of least privilege** (the DB user the application uses should only have SELECT/INSERT/UPDATE — never DDL or DROP privileges). Never concatenate user input directly into SQL strings.

Q33. What is the difference between TRUNCATE and DELETE? Can you roll back TRUNCATE?

`DELETE` removes rows one-by-one matching a WHERE clause, writes individual undo log entries for each row (enabling row-level rollback), fires row-level triggers, and is fully transactional. `TRUNCATE` removes ALL rows by deallocating the data pages, doesn't fire row-level triggers, doesn't write per-row undo log entries, and is dramatically faster for large tables (O(1) vs O(n)). In **PostgreSQL**, `TRUNCATE` IS transactional — it can be rolled back within an open transaction. In **MySQL**, `TRUNCATE` is NOT transactional and cannot be rolled back — it causes an implicit COMMIT before execution. This difference is a common trick question. Use TRUNCATE in PostgreSQL when you need to clear a large table during a transaction that might need to be rolled back (e.g., ETL load with rollback on failure).

Q34. What is a Composite Key? When is it used?

A **composite key** (or compound key) is a primary key that consists of two or more columns together. The combination of values across these columns must be unique,

even though individual column values may repeat. In MediBook, the `prescriptions` table has `PRIMARY KEY (appointment_id, prescription_id)` — prescription #1 for appointment A1 and prescription #1 for appointment A2 are different records. The `doctor_specializations` table uses `PRIMARY KEY (doctor_id, specialization_id)` — a doctor can be linked to a specialization exactly once. Composite keys arise naturally in junction tables for M:N relationships and in weak entity tables. A key consideration: columns in composite primary keys should be chosen carefully, as they may be used as foreign keys in child tables — requiring all component columns to be included in the FK.

Q35. What is a Self-Join? Provide a use case.

A **self-join** joins a table to itself, treating the same table as if it were two different tables by using aliases. It is used when rows in a table have a relationship with OTHER rows in the same table. **Use case in MediBook:** A `doctor_referrals` table has `referrer_doctor_id` and `referred_doctor_id`. To find pairs of doctors who refer to each other mutually:

```sql
SELECT a.referrer_doctor_id, b.referrer_doctor_id
FROM doctor_referrals a
JOIN doctor_referrals b ON a.referrer_doctor_id = b.referred_doctor_id
                       AND a.referred_doctor_id = b.referrer_doctor_id;
```

Other use cases: finding employees and their managers (both in an `employees` table with `manager_id`), comparing prices of products to their category averages, finding all patients who share the same birthday.

Q36. What is a Sequence and how is it used in PostgreSQL?

A **sequence** is a database object that generates a sequence of unique integer values, primarily used for auto-incrementing primary keys. `SERIAL` in PostgreSQL is syntactic sugar that creates an integer column backed by an auto-created sequence. You can also create sequences explicitly: `CREATE SEQUENCE order_id_seq START 1000 INCREMENT BY 1;` and use `NEXTVAL('order_id_seq')` in your insert. Sequences are designed to be non-blocking — multiple transactions can request the next value simultaneously without waiting for each other. However, this means sequence values may have **gaps** (if a transaction generates a sequence value but then rolls back). Never use sequences as a "count" of records — they tell you nothing about how many rows actually exist; they only guarantee uniqueness.

Q37. Explain how PostgreSQL handles Write-Ahead Logging (WAL).

WAL is PostgreSQL's mechanism for ensuring durability — the "D" in ACID. Before any modification is written to the actual data files (heap pages, index pages), the change is first written to the WAL — a sequential log file. The WAL entry records the before-image and after-image of every change. When a transaction COMMITs, the WAL is flushed to disk (fsync'd) before the client receives the success response. This means that even if the system crashes milliseconds after a COMMIT, the WAL on disk allows complete recovery. On restart after a crash, PostgreSQL's recovery process reads the WAL from the last checkpoint, redoes all changes (including those not yet in data files), then undoes changes from any uncommitted transactions. WAL also enables PostgreSQL's streaming replication — replicas apply the same WAL stream to stay in sync.

Q38. What is the difference between Optimistic and Pessimistic Locking?

Pessimistic locking assumes conflicts WILL happen and locks the data upfront. Before reading a row you intend to update, you acquire an exclusive lock (`SELECT ... FOR UPDATE`) so no other transaction can modify it until you're done. This prevents conflicts but reduces concurrency — other transactions must wait. Best for high-contention scenarios where conflicts are frequent and the cost of retrying is high. **Optimistic locking** assumes conflicts are RARE. It reads data without locking, performs changes, then at commit time checks if the data was modified by someone else (typically via a version number or timestamp column). If a conflict is detected, the transaction is retried. This maximizes concurrency but requires retry logic. Best for low-contention scenarios. In PostgreSQL, Serializable Snapshot Isolation (SSI) provides optimistic conflict detection without explicit version columns.

Q39. What is Partitioning in PostgreSQL? What types exist?

Table partitioning divides a logically single large table into smaller physical pieces (partitions) stored as separate sub-tables, while appearing as one unified table to queries. PostgreSQL supports three native partition types: **Range partitioning** — divides data based on value ranges of a column (e.g., appointments by year/month); **List partitioning** — divides based on explicit discrete values (e.g., patients by country: 'IN', 'US', 'UK'); **Hash partitioning** — distributes rows using a hash function for even distribution when no natural range or list exists. The major benefit is **partition pruning** — PostgreSQL's query planner reads only relevant partitions. A query `WHERE appointment_date = '2024-06-15'` on a date-range-partitioned table scans only the `appointments_2024` partition, not 5 years of data. This can turn minute-long queries into millisecond-long ones.

Q40. What are phantom reads? How does the Serializable isolation level prevent them?

A **phantom read** occurs when a transaction re-executes a range query and finds new rows that didn't exist when the query first ran — "phantom" rows that appeared between the two reads. Example: T1 counts available appointment slots (gets 5), T2 inserts a new booking and commits, T1 recounts and gets 4. The first count "saw" 5 slots; that 5th slot is now a phantom. Standard **Repeatable Read** locks individual ROWS that were read, preventing their modification — but new rows matching the same criteria can still be inserted. **Serializable** isolation prevents phantom reads by detecting read-write conflicts between transactions at the "predicate" level — if T1 reads a range and T2 writes to that range, the serialization checker detects a conflict and aborts one transaction. PostgreSQL implements this via Serializable Snapshot Isolation (SSI) using predicate locking.

Q41. What is the difference between Primary and Secondary indexing in terms of performance?

A **primary (clustered) index** physically orders the table data by the index key, so consecutive key values are on adjacent disk pages. Range scans on the primary key are extremely fast because after finding the starting page, subsequent rows are on the next physical pages — sequential I/O. A **secondary (non-clustered) index** stores pointers back to the actual rows, which may be scattered randomly across many different disk pages. For a query that returns many rows from a secondary index, each row may require a random I/O to a different page — potentially slower than a full table scan if many rows match (this is why the optimizer may ignore a secondary index for low-selectivity queries). The optimizer uses **index selectivity** to decide whether to use an index: if only 1% of rows match, the index is used; if 60% match, a sequential scan may be faster.

Q42. What is a Covering Index?

A **covering index** is an index that includes all the columns required to answer a specific query — so the database can read all needed data directly from the index without touching the main table (heap) at all. This is called an **index-only scan** and is significantly faster because it avoids the extra I/O of fetching actual table rows. In PostgreSQL, you can create covering indexes using the `INCLUDE` clause:

```sql
```

```
-- For the frequent query: SELECT doctor_id, appointment_date, status FROM appoi
CREATE INDEX idx_covering_appt ON appointments(doctor_id) INCLUDE (appointment_d
```

The `doctor_id` is the search key; `appointment_date` and `status` are included in the leaf nodes but not used for searching. The query can be satisfied entirely from the index. Covering indexes trade extra index size and write overhead for dramatically faster reads on specific queries.

Q43. How does GROUP BY work with HAVING? What is the processing order?

`GROUP BY` collects all rows with the same values in the specified columns into a single group and computes aggregate functions for each group. `HAVING` then filters these groups based on a condition — typically involving aggregate functions. The processing order within a query: `FROM/JOIN → WHERE (filter individual rows) → GROUP BY (form groups) → HAVING (filter groups) → SELECT (compute window functions, aliases) → ORDER BY → LIMIT`. A crucial implication: you can reference aggregate functions in HAVING (`HAVING AVG(consultation_fee) > 1000`) but NOT in WHERE. Also: every non-aggregated column in SELECT must appear in GROUP BY (or be functionally dependent on GROUP BY columns). PostgreSQL's implementation is stricter than MySQL's — it enforces this rule, which catches bugs early.

Q44. What is an ERD (Entity Relationship Diagram) and why is it important before writing SQL?

An ERD is a visual blueprint of a database schema that shows entities (tables), their attributes (columns), and the relationships between them, including cardinality (1:1, 1:N, M:N) and participation constraints (total/partial). Creating an ERD BEFORE writing SQL is important for several reasons: it forces you to identify all entities and their relationships correctly (catching mistakes like missing junction tables for M:N relationships), it communicates the schema to all stakeholders (developers, product managers, business analysts) in a universally understood visual language, and it makes normalization issues visible (if an entity's attribute is really a property of another entity, the ERD makes this clear). Skipping the ERD and jumping straight to SQL often results in schemas that need painful restructuring after real-world usage reveals the design flaws.

Q45. What is the use of COALESCE and NULLIF in SQL?

`COALESCE(a, b, c, ...)` returns the first non-NULL argument from its list. It is the

standard way to replace NULL with a default value: `COALESCE(consultation_notes, 'No notes recorded')` returns the notes if present, otherwise the default string. It's also used to merge NULL columns from OUTER JOINs. `NULLIF(a, b)` returns NULL if a = b; otherwise returns a. It's primarily used to prevent **division by zero**: `SUM(revenue) / NULLIF(COUNT(*), 0)` — if count is 0, NULLIF returns NULL, making the division NULL rather than raising an error. Both functions are non-short-circuit in terms of evaluation order (all arguments are evaluated) but are NULL-safe, making them essential tools for defensive SQL writing.

Q46. What is the difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column)?

`COUNT(*)` counts ALL rows in the group, including rows where all columns are NULL. It is the fastest form and is used when you want the total row count. `COUNT(column_name)` counts rows where `column_name` is NOT NULL — it ignores NULL values in that column. This is useful for counting "how many appointments have consultation notes." `COUNT(DISTINCT column_name)` counts the number of UNIQUE non-NULL values in that column. Example: `COUNT(DISTINCT doctor_id)` on the appointments table gives the number of unique doctors who had appointments — very different from `COUNT(doctor_id)` which counts total appointments (with duplicates). Performance note: `COUNT(DISTINCT ...)` is more expensive because it requires deduplication; for large tables, consider using approximate counting functions like `approx_count_distinct` if PostgreSQL HyperLogLog extensions are available.

Q47. How do you find and eliminate duplicate rows in a table?

Duplicate rows are rows that have identical values in all (or a meaningful subset of) columns. To find them, use GROUP BY and HAVING:

```sql
SELECT patient_id, first_name, last_name, email, COUNT(*) AS duplicate_count
FROM patients
GROUP BY patient_id, first_name, last_name, email
HAVING COUNT(*) > 1;
```

To delete duplicates while keeping one row (the one with the smallest `ctid`, PostgreSQL's physical row ID):

```sql
```

```sql
DELETE FROM patients
WHERE ctid NOT IN (
    SELECT MIN(ctid) FROM patients
    GROUP BY first_name, last_name, email
);
```

A better long-term solution is to add UNIQUE constraints on the relevant columns so duplicates are rejected at insert time. For very large tables, a CTE approach with `ROW_NUMBER()` partitioned by the duplicate key is often cleaner and faster than the NOT IN approach.

Q48. What is a Recursive CTE? Provide a use case.

A **recursive CTE** (Common Table Expression) is a CTE that references itself, allowing you to process hierarchical or graph-structured data in SQL without procedural code. The structure has two parts: a **base case** (non-recursive SELECT that defines the starting rows) and a **recursive case** (SELECT that references the CTE itself, building up the result). PostgreSQL requires a depth/cycle guard to prevent infinite recursion. **Real use case:** An `employees` table with `(employee_id, manager_id)` — the hierarchy can be arbitrarily deep. A recursive CTE finds all subordinates of a given manager: start with the manager (base case), then in each recursive iteration add employees whose `manager_id` matches employees found so far. This is functionally equivalent to a BFS (Breadth-First Search) on the org chart hierarchy. The same technique applies to category hierarchies, filesystem directories, and network topology traversal.

Q49. What is the difference between a Stored Procedure and an Application-Layer Query?

A **stored procedure** is precompiled SQL code stored in the database itself, executed on the database server. An **application-layer query** is SQL text sent from the application server to the database server at runtime, parsed and planned on each execution (mitigated by prepared statements which are parsed once). Stored procedures offer: **reduced network traffic** (one call instead of many round-trips), **precompiled execution plans** (no per-call parsing overhead), **encapsulation of business logic** in the database (accessible to any client language), and **enhanced security** (grant EXECUTE on a procedure without granting table-level access). Downsides: **harder to version control and test** (code lives in the database, not your Git repo), **tighter coupling** between application and database, **vendor lock-in** (PL/pgSQL is PostgreSQL-specific). Modern ORMs and connection poolers have reduced the performance gap, making application-layer queries with prepared statements competitive.

Q50. How would you design a database schema for a system like MediBook from scratch?

Start with **requirements gathering**: identify the main entities (Patient, Doctor, Hospital, Appointment, Prescription, Specialization), understand cardinalities (one patient → many appointments, one doctor → many appointments, M:N between doctors and specializations), and identify key business rules (no double-booking, each prescription belongs to an appointment). Create an **ER diagram** first — visualize all entities, their attributes, and relationships. **Apply normalization**: ensure each table is in BCNF — no redundant doctor names stored in appointment rows, no hospital names stored in doctor rows. **Define constraints**: PRIMARY KEYs on every table, FOREIGN KEYs for all relationships, UNIQUE constraints on business keys (doctor email, appointment (doctor, date, time)), CHECK constraints for enum-like columns (status IN ('scheduled','completed','cancelled')). **Add indexes** on FK columns and frequently-queried columns. **Consider partitioning** early for time-series tables like `appointments`. **Plan for scale**: identify read-heavy vs. write-heavy patterns to decide on replication, caching (Redis for slot availability), and which materialized views to pre-compute. Document every decision — schema design is as much communication as technical art.

---

9. Ultimate LeetCode SQL Hitlist

> A curated list of 30 essential LeetCode SQL problems covering every major concept tested in placement interviews. Sorted by concept category.

#	Problem ID	Problem Name	Link	Concept Tested
1	175	Combine Two Tables	Link	LEFT JOIN — returning rows even when related table has no match
2	176	Second Highest Salary	Link	OFFSET/LIMIT, subquery with MAX, handling NULL when value doesn't exist
3	177	Nth Highest Salary	Link	Parameterized function, OFFSET, handling edge cases
4	178	Rank Scores	Link	DENSE_RANK window function, ORDER BY within OVER()
5	180	Consecutive Numbers	Link	Self-Join three times, or LAG/LEAD window functions
6	181	Employees Earning More Than Their Managers	Link	Self-Join on same table with different aliases
7	182	Duplicate Emails	Link	GROUP BY + HAVING COUNT(*) > 1
8	183	Customers Who Never Order	Link	LEFT JOIN with IS NULL, or NOT IN subquery, or NOT EXISTS
9	184	Department Highest Salary	Link	Correlated subquery or window function with RANK/DENSE_RANK
10	185	Department Top Three Salaries	Link	DENSE_RANK() with PARTITION BY, filtering Top-N per group
11	196	Delete Duplicate Emails	Link	DELETE with self-join, keeping minimum ID per group
12	197	Rising Temperature	Link	Self-Join or LAG on date arithmetic, comparing consecutive rows
13	262	Trips and Users	Link	JOIN with filtering, conditional aggregation (CASE + AVG), date range filtering
14	511	Game Play Analysis I	Link	MIN() aggregate, GROUP BY to find first event per user

#	Problem ID	Problem Name	Link	Concept Tested
15	512	Game Play Analysis II	Link	Correlated subquery or window function to find first device per user
16	534	Game Play Analysis III	Link	Running sum with SUM() OVER (PARTITION BY ORDER BY) window function
17	550	Game Play Analysis IV	Link	Comparing date with MIN date + 1, EXISTS or JOIN for consecutive day check
18	569	Median Employee Salary	Link	PERCENTILE_CONT or ROW_NUMBER with complex median logic
19	570	Managers with at Least 5 Direct Reports	Link	Self-join on manager_id, HAVING COUNT >= 5
20	574	Winning Candidate	Link	Joining aggregated subquery, finding MAX COUNT group
21	577	Employee Bonus	Link	LEFT JOIN + handling NULL (bonus < 1000 OR bonus IS NULL)
22	578	Get Highest Answer Rate Question	Link	Conditional COUNT (CASE WHEN), calculating rates, LIMIT 1
23	579	Find Cumulative Salary of an Employee	Link	SUM OVER sliding window, excluding current month's latest entry
24	580	Count Student Number in Departments	Link	LEFT JOIN to include departments with 0 students, COUNT with NULL handling
25	585	Investments in 2016	Link	Multi-condition WHERE with subqueries, EXISTS and NOT EXISTS together
26	601	Human Traffic of Stadium	Link	Consecutive rows with count >= 100, complex self-join or window functions
27	602	Friend Requests II: Who Has the Most Friends	Link	UNION ALL (bi-directional edges), GROUP BY, finding MAX
28	615	Average Salary: Departments vs. Company	Link	Comparing group average to overall average using window functions

#	Problem ID	Problem Name	Link	Concept Tested
29	618	Students Report by Geography	Link	Pivot table using conditional aggregation (CASE WHEN + ROW_NUMBER)
30	1045	Customers Who Bought All Products	Link	HAVING COUNT(DISTINCT) = total products, implementing the DIVISION operator

Additional High-Value Problems (Recommended)

#	Problem ID	Problem Name	Link	Concept Tested
31	1070	Product Sales Analysis III	Link	Filtering with correlated subquery, finding first year per product
32	1084	Sales Analysis III	Link	Exclusive period filtering using HAVING with MIN/MAX date bounds
33	1193	Monthly Transactions I	Link	Date formatting (DATE_FORMAT/TO_CHAR), conditional COUNT with CASE WHEN
34	1204	Last Person to Fit in the Bus	Link	Running SUM window function, filtering with HAVING or subquery
35	1321	Restaurant Growth	Link	7-day sliding window SUM/AVG using window function with ROWS BETWEEN

10. Bonus: Extra Placement Topics

10.1 Stored Procedures & Functions

Stored procedures and functions are reusable, named blocks of SQL and procedural logic stored in the database. They offer modularity, performance benefits (single network call vs. many), and security (EXECUTE permission without table access).

PostgreSQL Function Example: Get Doctor Availability

```sql
```

```sql
-- Function that returns available time slots for a doctor on a given date:
CREATE OR REPLACE FUNCTION get_available_slots(
    p_doctor_id     INT,
    p_date          DATE,
    p_slot_interval INTERVAL DEFAULT '30 minutes'
)
RETURNS TABLE(available_time TIME) AS $$
DECLARE
    clinic_start TIME := '09:00:00';
    clinic_end   TIME := '17:00:00';
    v_current    TIME;
BEGIN
    v_current := clinic_start;
    WHILE v_current < clinic_end LOOP
        -- Check if this slot is NOT already booked
        IF NOT EXISTS (
            SELECT 1 FROM appointments a
            WHERE a.doctor_id = p_doctor_id
              AND a.appointment_date = p_date
              AND a.appointment_time = v_current
              AND a.status NOT IN ('cancelled')
        ) THEN
            available_time := v_current;
            RETURN NEXT;
        END IF;
        v_current := v_current + p_slot_interval;
    END LOOP;
END;
$$ LANGUAGE plpgsql STABLE;

-- Usage:
SELECT * FROM get_available_slots(7, '2025-03-15');
SELECT * FROM get_available_slots(7, '2025-03-15', '1 hour');
```

PostgreSQL Procedure Example: Cancel Appointment with Refund

```sql
```

```sql
CREATE OR REPLACE PROCEDURE cancel_appointment_with_refund(
    p_appointment_id INT,
    p_cancelled_by   VARCHAR(50)
)
LANGUAGE plpgsql AS $$
DECLARE
    v_patient_id    INT;
    v_doctor_fee    NUMERIC;
    v_appt_date     DATE;
    v_appt_status   VARCHAR(20);
BEGIN
    -- Get appointment details
    SELECT a.patient_id, d.consultation_fee, a.appointment_date, a.status
    INTO v_patient_id, v_doctor_fee, v_appt_date, v_appt_status
    FROM appointments a
    JOIN doctors d ON d.doctor_id = a.doctor_id
    WHERE a.appointment_id = p_appointment_id
    FOR UPDATE;  -- Lock the row

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Appointment % not found', p_appointment_id;
    END IF;

    IF v_appt_status = 'completed' THEN
        RAISE EXCEPTION 'Cannot cancel a completed appointment';
    END IF;

    IF v_appt_status = 'cancelled' THEN
        RAISE EXCEPTION 'Appointment already cancelled';
    END IF;

    -- Cancel the appointment
    UPDATE appointments
    SET status = 'cancelled'
    WHERE appointment_id = p_appointment_id;

    -- Refund full amount if cancelled 24+ hours in advance
    IF v_appt_date - CURRENT_DATE >= 1 THEN
        UPDATE patient_wallets
        SET balance = balance + v_doctor_fee
        WHERE patient_id = v_patient_id;
        RAISE NOTICE 'Full refund of % issued to patient %', v_doctor_fee, v_pati
    ELSE
        RAISE NOTICE 'No refund - cancellation within 24 hours';
```

```
    END IF;

    COMMIT;
END;
$$;

-- Usage:
CALL cancel_appointment_with_refund(42, 'receptionist_user');
```

10.2 Database Security & SQL Injection

Role-Based Access Control in PostgreSQL

sql

```
-- Create role hierarchy:
CREATE ROLE medibook_readonly;
CREATE ROLE medibook_receptionist;
CREATE ROLE medibook_doctor;
CREATE ROLE medibook_admin;

-- Grant permissions at the right level:
GRANT SELECT ON ALL TABLES IN SCHEMA public TO medibook_readonly;

GRANT SELECT ON patients, appointments, doctors TO medibook_receptionist;
GRANT INSERT, UPDATE ON appointments TO medibook_receptionist;

GRANT SELECT ON patients, appointments, prescriptions, lab_tests TO medibook_doc
GRANT INSERT, UPDATE ON prescriptions, lab_tests TO medibook_doctor;
-- Doctors can only UPDATE their own appointments (use Row Level Security):

-- Row Level Security (RLS) — doctor can only view their own appointments:
ALTER TABLE appointments ENABLE ROW LEVEL SECURITY;

CREATE POLICY doctor_own_appointments ON appointments
    FOR ALL
    TO medibook_doctor
    USING (doctor_id = current_setting('app.current_doctor_id')::INT);

-- Admin gets everything:
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO medibook_admin;

-- Assign roles to actual user accounts:
CREATE USER dr_sharma LOGIN PASSWORD 'hashedpass';
GRANT medibook_doctor TO dr_sharma;
```

Preventing SQL Injection (Application Layer)

python

```
# BAD - vulnerable to SQL injection:
query = f"SELECT * FROM patients WHERE email = '{user_input}'"
# User input: "'; DROP TABLE patients; --" → catastrophic!

# GOOD - parameterized query (Python psycopg2):
cursor.execute("SELECT * FROM patients WHERE email = %s", (user_input,))

# GOOD - using an ORM (SQLAlchemy):
patient = session.query(Patient).filter(Patient.email == user_input).first()
```

10.3 Partitioning in PostgreSQL

Declarative Partitioning Full Example

sql

```
-- Range Partitioning on appointments by year:
CREATE TABLE appointments (
    appointment_id   BIGSERIAL,
    patient_id       INT NOT NULL,
    doctor_id        INT NOT NULL,
    appointment_date DATE NOT NULL,
    status           VARCHAR(20),
    PRIMARY KEY (appointment_id, appointment_date)  -- partition key must be in P
) PARTITION BY RANGE (appointment_date);

CREATE TABLE appointments_2023 PARTITION OF appointments
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

CREATE TABLE appointments_2024 PARTITION OF appointments
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

CREATE TABLE appointments_2025 PARTITION OF appointments
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

-- Default partition catches anything not matched by above:
CREATE TABLE appointments_default PARTITION OF appointments DEFAULT;

-- Create indexes on each partition (or on parent, which propagates):
CREATE INDEX ON appointments(doctor_id);
CREATE INDEX ON appointments(appointment_date);

-- Verify partition pruning in EXPLAIN:
EXPLAIN SELECT * FROM appointments WHERE appointment_date = '2024-06-01';
-- Should show: "Partitions selected: appointments_2024" (only one partition scan
```

10.4 Query Optimization Cheat Sheet

The EXPLAIN Output Decoder

When you run `EXPLAIN (ANALYZE, BUFFERS) query`, the output contains crucial information:

```
Index Scan using idx_appts_doctor_date on appointments (cost=0.43..8.45 rows=1
width=85)
    (actual time=0.028..0.030 rows=1 loops=1)
  Index Cond: ((doctor_id = 5) AND (appointment_date = '2024-06-01'))
  Buffers: shared hit=4 read=0
```

Term	Meaning
`cost=0.43..8.45`	Estimated cost: startup cost .. total cost (in arbitrary units)
`rows=1`	Estimated number of rows output
`actual time=0.028..0.030`	Real time: first row .. last row (milliseconds)
`loops=1`	How many times this node executed
`Buffers: shared hit=4`	Pages found in cache (good); `read=4` means disk reads (bad)
`Seq Scan`	Full table scan — look for missing index
`Index Scan`	Using B+ Tree index — efficient for selective queries
`Bitmap Index Scan`	Building a bitmap, then heap access — for multiple indexes
`Hash Join`	Builds hash table from smaller table — good for large tables
`Nested Loop`	For each row of outer, probe inner — good when inner is small
`Merge Join`	Both inputs sorted — good when sorting is pre-done

Top 10 Query Optimization Techniques

1. **Always run EXPLAIN ANALYZE before and after any optimization** — measure, don't guess.
2. **Index every foreign key column** — unindexed FK columns cause full scans on joins.
3. **Use composite indexes wisely** — most selective column first; consider covering indexes.
4. **Avoid functions on indexed columns in WHERE** — `WHERE UPPER(email) = 'X'` ignores the index on `email`. Use `expression indexes` or normalize data before storing.

5. **Use LIMIT early** — if you only need the top 10 results, let the query planner know.
6. **Rewrite correlated subqueries as JOINs or window functions** — correlated subqueries run N times; joins and window functions are usually set-based and much faster.
7. **Keep statistics up to date** — `ANALYZE tablename` refreshes planner statistics so it makes better choices. Run after large bulk loads.
8. **Partition large time-series tables** — enables partition pruning, dramatically reducing data scanned.
9. **Use MATERIALIZED VIEWS for expensive, frequently-run aggregations** — refresh nightly or on schedule.
10. **Avoid SELECT * in production queries** — explicitly list columns; this enables covering indexes, reduces network payload, and prevents schema changes from breaking queries.

```sql
-- Expression index example (to allow indexing on UPPER(email)):
CREATE INDEX idx_patients_email_upper ON patients(UPPER(email));
-- Now this query CAN use the index:
SELECT * FROM patients WHERE UPPER(email) = 'RIYA@EXAMPLE.COM';

-- Partial index example (index only scheduled appointments — much smaller):
CREATE INDEX idx_appts_scheduled ON appointments(doctor_id, appointment_date)
WHERE status = 'scheduled';
-- Only queries with WHERE status = 'scheduled' can use this index, but it's tiny
```

End of the Ultimate DBMS & SQL Placement Preparation Guide

> **Quick Reference Summary:**
>
> - Normalization path: 1NF → 2NF (fix partial deps) → 3NF (fix transitive deps) → BCNF (fix all non-superkey determinants) → 4NF (fix MVDs) → 5NF (fix join deps)
> - Isolation levels: READ UNCOMMITTED < READ COMMITTED (default PG) < REPEATABLE READ < SERIALIZABLE
> - Index types: B+ Tree (default, range-friendly), Hash (equality only), Bitmap (low cardinality, multi-column filters)
> - CAP: You get at most 2 of 3; since P is always required, real choice is CP vs AP

- ACID is for single-node; BASE (Basically Available, Soft state, Eventually consistent) is the NoSQL equivalent
- B+ Tree advantage over B-Tree: all data in leaf nodes + linked list = faster range scans + better cache usage